

INSTITUTO INFNET

NEUZA GIOVANNA DE OLIVEIRA

DISCIPLINA: Projeto de Bloco: Desenvolvimento Back-End

Mudança de Requisitos, Especificação e Projeto

MediCare — Plataforma de Agendamento e Gerenciamento de Consultas Médicas

---

1. Introdução

Na terceira etapa foram modelados as classes, os relacionamentos e o dicionário de dados do MediCare, e foi iniciada a implementação da leitura e escrita dos dados em arquivos CSV. Ao revisar os dados que serão manipulados pelo sistema, o cliente identificou necessidades que não haviam sido levantadas nos requisitos iniciais.

Esta etapa corresponde à fase de Construção do RUP. O documento registra as mudanças solicitadas e o impacto de cada uma, revisa os casos de uso, atualiza o diagrama de classes e o dicionário de dados, apresenta os diagramas de sequência, lista os métodos de cada classe e descreve como a implementação foi atualizada.

---

2. Mudanças de Requisitos

As mudanças solicitadas pelo cliente estão resumidas abaixo, junto com o impacto de cada uma no projeto.

Mudança	O que o cliente pediu	Impacto no projeto
M1 — Recepcionista	Funcionários da recepção devem poder consultar a disponibilidade, agendar, remarcar e cancelar consultas em nome dos pacientes que são atendidos no balcão.	Novo ator e nova classe Recepcionista. Os casos de uso UC03 a UC07 passam a ter a recepcionista como ator.
M2 — Notificações	O paciente deve receber confirmações e lembretes das consultas por e-mail, SMS ou WhatsApp, por meio de um sistema de notificação externo.	Novo ator externo (Sistema de Notificação), nova classe Notificacao e novo caso de uso UC14. O paciente passa a informar o canal preferido. UC05, UC06 e UC07 passam a gerar notificações.
M3 — Histórico do paciente	Durante o atendimento, o médico precisa consultar as consultas anteriores do paciente.	Novo caso de uso UC15. UC10 passa a permitir registrar a consulta como realizada, o que alimenta o histórico.

Mudança	O que o cliente pediu	Impacto no projeto
M4 — Login com conta externa	O usuário deve poder entrar com uma conta Google, Microsoft ou Meta, sem criar senha na plataforma.	Novo ator externo (Sistema de autenticação SSO) e novo atributo provedor em Usuario. UC02 recebe um fluxo alternativo. A integração com os provedores fica para uma etapa posterior; nesta entrega o modelo e o tratamento no login já estão preparados.

2.1 Requisitos funcionais incluídos

Os requisitos da TP1 continuam válidos. Foram incluídos:

- O sistema deve permitir que a recepcionista consulte horários, agende, reagende e cancele consultas de pacientes cadastrados.
- O sistema deve permitir que o paciente escolha o canal pelo qual recebe avisos (e-mail, SMS ou WhatsApp).
- O sistema deve gerar uma notificação de confirmação ao agendar, de reagendamento ao reagendar e de cancelamento ao cancelar uma consulta.
- O sistema deve gerar lembretes das consultas agendadas para o dia seguinte e entregá-los ao sistema de notificação.
- O sistema deve permitir que o médico registre uma consulta como realizada.
- O sistema deve permitir que o médico consulte o histórico de consultas realizadas de um paciente que ele atende.
- O sistema deve permitir que o usuário se autentique com uma conta Google, Microsoft ou Meta.

---

3. Revisão dos Casos de Uso

3.1 Atores

Ator	Descrição	Situação
Paciente	Pesquisa médicos, agenda, cancela e reagenda consultas e recebe notificações.	Alterado
Recepcionista	Funcionário da recepção que consulta horários e agenda, reagenda e cancela consultas em nome dos pacientes.	Novo
Médico	Gerencia disponibilidade e agenda, registra consultas realizadas e consulta o histórico dos pacientes que atende.	Alterado
Administrador	Gerencia usuários e agendamentos e aciona o envio das notificações.	Alterado
Sistema de Notificação	Sistema externo que recebe os dados da consulta e entrega confirmações e lembretes por e-mail, SMS ou WhatsApp.	Novo
Sistema de autenticação SSO	Sistema externo (Google, Microsoft, Meta) que informa ao MediCare os dados do usuário autenticado.	Novo

3.2 Lista de casos de uso

ID	Caso de uso	Atores	Situação
UC01	Criar conta	Paciente, Médico	Alterado
UC02	Realizar login	Todos os usuários, Sistema SSO	Alterado
UC03	Pesquisar médicos	Paciente, Recepcionista	Alterado
UC04	Consultar horários disponíveis	Paciente, Recepcionista	Alterado
UC05	Agendar consulta	Paciente, Recepcionista	Alterado
UC06	Cancelar consulta	Paciente, Recepcionista	Alterado
UC07	Reagendar consulta	Paciente, Recepcionista	Alterado
UC08	Gerenciar perfil	Paciente, Médico	Alterado
UC09	Gerenciar disponibilidade	Médico	Alterado
UC10	Gerenciar agenda	Médico	Alterado
UC11	Realizar teleconsulta	Paciente, Médico	Alterado
UC12	Gerenciar usuários	Administrador	Alterado
UC13	Gerenciar agendamentos	Administrador	Alterado
UC14	Enviar notificações	Administrador, Sistema de Notificação	Novo
UC15	Consultar histórico do paciente	Médico	Novo

3.3 Descrição dos casos de uso

A seguir estão os casos de uso reescritos e os novos.

UC01 — Criar conta (alterado)

Atores: Paciente, Médico

Objetivo: Permitir que um novo paciente ou médico seja cadastrado na plataforma.

Fluxo principal:

1. O usuário seleciona a opção de criação de conta.
2. O sistema solicita os dados de cadastro. O paciente informa nome, e-mail, senha, telefone, data de nascimento e o canal em que deseja receber avisos. O médico informa nome, e-mail, senha, CRM, telefone e especialidade.
3. O sistema verifica se o e-mail já está em uso por qualquer usuário.
4. O sistema registra a conta com a senha armazenada em forma de hash.
5. O sistema informa que o cadastro foi realizado.

Fluxos alternativos:

- 3a. E-mail já cadastrado: o sistema informa o erro e não cria a conta.

UC02 — Realizar login (alterado)

Atores: Paciente, Recepcionista, Médico, Administrador; Sistema de autenticação SSO

Objetivo: Permitir que o usuário acesse as funcionalidades do seu perfil.

Fluxo principal:

1. O usuário informa e-mail e senha.
2. O sistema localiza a conta pelo e-mail.
3. O sistema verifica a senha e se a conta está ativa.
4. O sistema apresenta o menu correspondente ao tipo de usuário.

Fluxos alternativos:

- 1a. Login com conta externa: o usuário escolhe entrar com Google, Microsoft ou Meta; o sistema de autenticação SSO valida o usuário e devolve seus dados; o sistema localiza a conta vinculada e segue para o passo 4.
- 2a. A conta encontrada utiliza provedor externo: o sistema orienta o usuário a entrar pelo provedor.
- 3a. Senha incorreta, e-mail inexistente ou conta desativada: o sistema informa o erro.

UC03 — Pesquisar médicos (alterado)

Atores: Paciente, Recepcionista

Objetivo: Permitir que o paciente, ou a recepcionista que o atende, encontre médicos na plataforma.

Fluxo principal:

1. O usuário acessa a pesquisa.
2. O usuário informa a especialidade desejada ou pesquisa todas.
3. O sistema apresenta os médicos ativos correspondentes.

UC04 — Consultar horários disponíveis (alterado)

Atores: Paciente, Recepcionista

Objetivo: Permitir a consulta dos horários livres de um médico.

Fluxo principal:

1. O usuário seleciona um médico.
2. O sistema apresenta os horários livres, ordenados por data e hora, com a modalidade e, quando presencial, o local do atendimento.

UC05 — Agendar consulta (alterado)

Atores: Paciente, Recepcionista; Sistema de Notificação

Objetivo: Registrar uma consulta em um horário disponível.

Fluxo principal:

1. Quando o ator é a recepcionista, ela informa o e-mail do paciente atendido.
2. O usuário seleciona o médico e o horário.
3. O sistema verifica se o paciente está ativo e se o horário está livre.
4. O sistema reserva o horário e registra a consulta.
5. O sistema gera uma notificação de confirmação no canal preferido do paciente.
6. O sistema apresenta a confirmação do agendamento.

Fluxos alternativos:

- 3a. Horário já reservado ou paciente desativado: o sistema informa o erro e nada é registrado.

UC06 — Cancelar consulta (alterado)

Atores: Paciente, Recepcionista; Sistema de Notificação

Objetivo: Cancelar uma consulta agendada.

Fluxo principal:

1. O sistema lista as consultas agendadas do paciente.
2. O usuário seleciona a consulta.
3. O sistema altera a consulta para cancelada e libera o horário.
4. O sistema gera uma notificação de cancelamento.

Fluxos alternativos:

- 2a. A consulta escolhida não pertence ao paciente ou não está agendada: o sistema informa o erro.

UC07 — Reagendar consulta (alterado)

Atores: Paciente, Recepcionista; Sistema de Notificação

Objetivo: Alterar o horário de uma consulta agendada.

Fluxo principal:

1. O usuário seleciona uma consulta agendada.
2. O sistema apresenta os horários livres do mesmo médico.
3. O usuário escolhe o novo horário.
4. O sistema reserva o novo horário, libera o anterior e atualiza a consulta.
5. O sistema gera uma notificação de reagendamento.

Fluxos alternativos:

- 3a. Horário de outro médico ou já reservado: o sistema informa o erro e mantém a consulta como estava.

UC08 — Gerenciar perfil (alterado)

Atores: Paciente, Médico

Objetivo: Permitir que o usuário mantenha seus dados atualizados.

O paciente pode alterar o telefone e o canal pelo qual recebe as notificações. O médico pode alterar o telefone profissional.

UC09 — Gerenciar disponibilidade (alterado)

Atores: Médico

Objetivo: Permitir que o médico cadastre e remova seus horários de atendimento.

Fluxo principal:

1. O médico informa data, hora de início, hora de término e modalidade.
2. Se a modalidade for presencial, o médico escolhe um dos seus locais de atendimento.
3. O sistema verifica se a data não é passada, se o término é posterior ao início e se o horário não conflita com outro do mesmo médico.
4. O sistema registra o horário, que passa a ser oferecido aos pacientes.

Fluxos alternativos:

- 3a. Dados inválidos ou conflito de horário: o sistema informa o erro.
- Remoção: o médico pode remover um horário que nunca foi usado em uma consulta.

UC10 — Gerenciar agenda (alterado)

Atores: Médico

Objetivo: Permitir que o médico acompanhe suas consultas e registre os atendimentos.

Fluxo principal:

1. O médico abre a agenda.
2. O sistema apresenta as consultas agendadas, ordenadas por data e hora.
3. Após o atendimento, o médico seleciona a consulta e a registra como realizada.

Fluxos alternativos:

- 3a. A consulta não pertence à agenda do médico: o sistema informa o erro.

UC11 — Realizar teleconsulta (alterado)

Atores: Paciente, Médico

Objetivo: Permitir que paciente e médico acessem uma consulta por teleconsulta.

Fluxo principal:

1. O usuário escolhe acessar uma teleconsulta.
2. O sistema lista as teleconsultas agendadas em que o usuário participa.
3. O usuário seleciona a consulta.
4. O sistema verifica se o usuário é o paciente ou o médico da consulta, se a modalidade é teleconsulta e se a consulta está agendada.
5. O sistema verifica se o momento atual está dentro do horário agendado.
6. O sistema libera o acesso e a consulta é realizada de forma remota.

Fluxos alternativos:

- 4a. Usuário não participa, consulta presencial ou não agendada: o sistema informa o erro.
- 5a. Fora do horário agendado: o sistema informa o horário em que a teleconsulta poderá ser acessada.

UC12 — Gerenciar usuários (alterado)

Atores: Administrador

Objetivo: Permitir que o administrador acompanhe e controle as contas da plataforma.

O administrador lista pacientes, médicos, recepcionistas e administradores e pode ativar ou desativar uma conta pelo e-mail. Uma conta desativada não consegue realizar login.

UC13 — Gerenciar agendamentos (alterado)

Atores: Administrador; Sistema de Notificação

Objetivo: Permitir que o administrador acompanhe e cancele agendamentos.

Fluxo principal:

1. O administrador lista os agendamentos, podendo filtrar por situação.
2. O sistema apresenta as consultas ordenadas por data e hora.
3. Se necessário, o administrador seleciona uma consulta agendada e a cancela.
4. O sistema segue o mesmo processo de UC06: cancela a consulta, libera o horário e gera a notificação de cancelamento.

UC14 — Enviar notificações (novo)

Atores: Administrador, Sistema de Notificação

Objetivo: Entregar ao sistema de notificação as confirmações e os lembretes de consulta.

Fluxo principal:

1. O sistema gera notificações automaticamente ao agendar, reagendar ou cancelar (UC05 a UC07).
2. O administrador solicita a geração dos lembretes do dia seguinte.
3. O sistema cria um lembrete para cada consulta agendada para aquele dia que ainda não tenha lembrete.
4. O administrador solicita o envio das notificações pendentes.
5. O sistema entrega canal, destino e mensagem ao sistema de notificação e marca cada notificação como enviada.

UC15 — Consultar histórico do paciente (novo)

Atores: Médico

Objetivo: Permitir que o médico veja as consultas anteriores de um paciente.

Fluxo principal:

1. O médico informa o e-mail do paciente.
2. O sistema verifica se o paciente possui ao menos uma consulta com esse médico.
3. O sistema apresenta as consultas realizadas do paciente, da mais recente para a mais antiga.

Fluxos alternativos:

- 2a. O paciente não possui consultas com o médico: o sistema não exibe o histórico.

3.4 Mapeamento entre atores e casos de uso

Caso de uso	Paciente	Recep.	Médico	Admin.	Notif.	SSO
UC01 — Criar conta	X		X			
UC02 — Realizar login	X	X	X	X		X
UC03 — Pesquisar médicos	X	X				
UC04 — Consultar horários disponíveis	X	X				
UC05 — Agendar consulta	X	X			X	
UC06 — Cancelar consulta	X	X			X	
UC07 — Reagendar consulta	X	X			X	
UC08 — Gerenciar perfil	X		X			
UC09 — Gerenciar disponibilidade			X			
UC10 — Gerenciar agenda			X			
UC11 — Realizar teleconsulta	X		X			
UC12 — Gerenciar usuários				X		
UC13 — Gerenciar agendamentos				X	X	
UC14 — Enviar notificações				X	X	
UC15 — Consultar histórico do paciente			X			

Recep. = Recepcionista; Admin. = Administrador; Notif. = Sistema de Notificação; SSO = Sistema de autenticação SSO.

---

4. Diagrama de Classes Atualizado

O diagrama da TP3 foi revisado para incluir as mudanças. As classes novas aparecem em laranja.

Alterações no modelo:

- **Recepcionista**: nova subclasse de Usuario, sem atributos próprios. Assim como o Administrador, atua por meio de operações e não tem associação estrutural com as outras classes.
- **Notificacao**: nova classe associada à Consulta (uma consulta gera zero ou mais notificações). Guarda tipo, canal, destino, mensagem e situação do envio.
- **Usuario**: novo atributo *provedor*, que indica se a conta usa senha própria (Local) ou login externo. Em contas externas a senha fica vazia.
- **Paciente**: novo atributo *canalPreferido*, usado para definir o canal e o destino das notificações.
- **HorarioDisponivel**: nova operação *conflitaCom()*, usada na validação de UC09.
- **Enumerações novas**: ProvedorAutenticacao, CanalNotificacao, TipoNotificacao e StatusNotificacao.

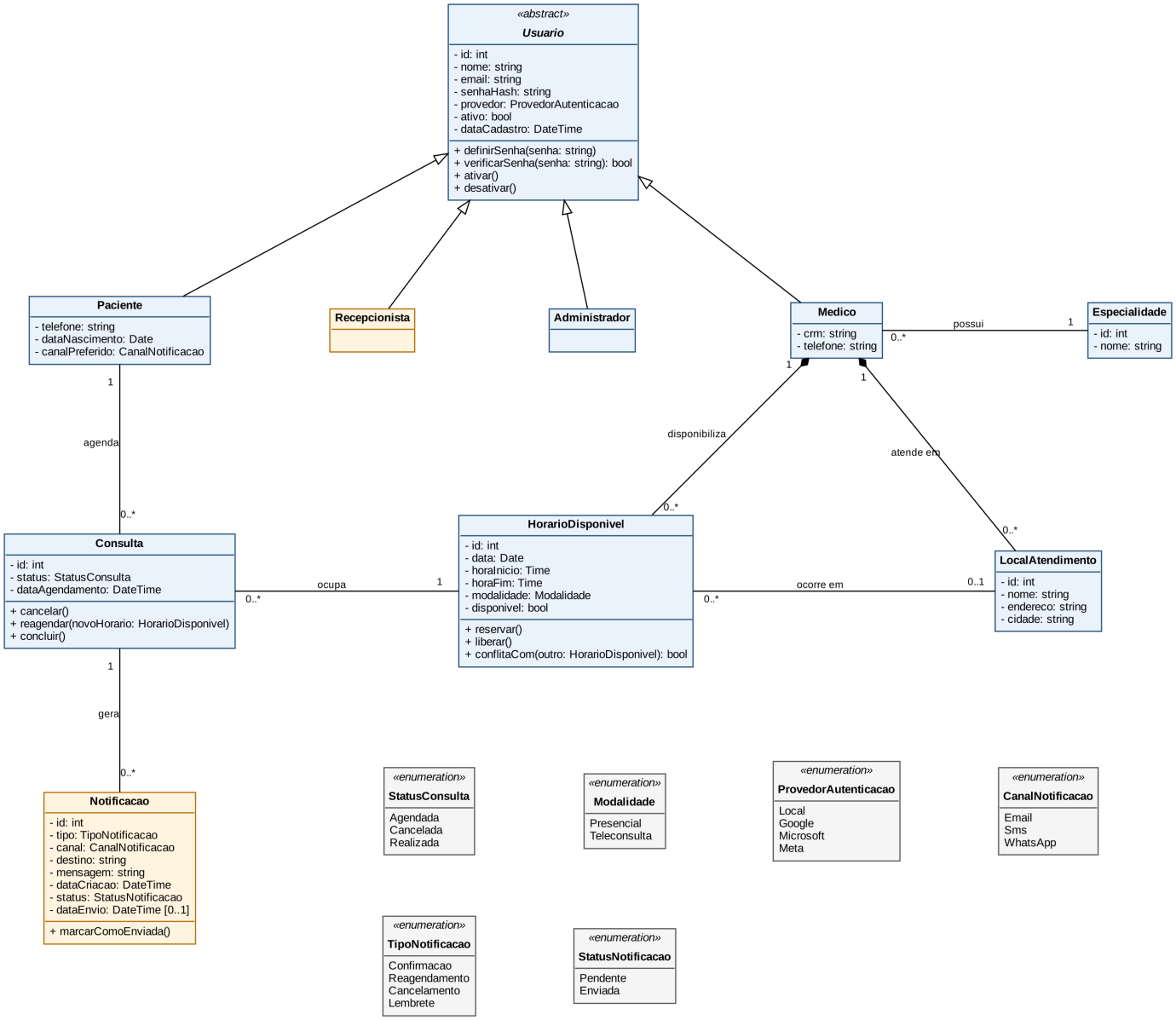


Figura 1 — Diagrama de Classes atualizado do sistema MediCare. Classes novas em laranja.

5. Dicionário de Dados Atualizado

5.1 Classes

Classe	Descrição	Situação
Usuario (abstrata)	Dados comuns de todos que acessam a plataforma.	Alterada
Paciente	Usuário que agenda consultas e recebe notificações.	Alterada
Recepcionista	Usuário da recepção que agenda, reagenda e cancela consultas para os pacientes.	Nova
Medico	Profissional que cadastra horários, atende e consulta o histórico dos pacientes.	Sem alteração
Administrador	Usuário que gerencia contas, agendamentos e o envio de notificações.	Sem alteração
Especialidade	Área de atuação usada na pesquisa de médicos.	Sem alteração
LocalAtendimento	Endereço de atendimento presencial de um médico.	Sem alteração
HorarioDisponivel	Intervalo oferecido pelo médico, com modalidade definida.	Alterada
Consulta	Agendamento de um paciente em um horário de um médico.	Sem alteração
Notificacao	Mensagem de confirmação, reagendamento, cancelamento ou lembrete a ser entregue ao paciente.	Nova

5.2 Atributos

As classes sem alteração mantêm os atributos descritos na TP3. Abaixo estão as classes novas e as alteradas, com os atributos novos indicados. Atributos de *referência* representam associações e, na implementação, guardam o Id do objeto relacionado.

Usuario

Atributo	Tipo	Obrig.	Descrição
id	int	Sim	Identificador único dentro do tipo de usuário.
nome	string	Sim	Nome completo.
email	string	Sim	E-mail de acesso; único entre todos os usuários.
senhaHash	string	Não	Hash SHA-256 da senha. Vazio quando o login é feito por provedor externo.
provedor (novo)	ProvedorAutenticacao	Sim	Local, Google, Microsoft ou Meta.
ativo	bool	Sim	Indica se a conta pode acessar a plataforma.
dataCadastro	DateTime	Sim	Data e hora de criação da conta.

Paciente (herda de Usuario)

Atributo	Tipo	Obrig.	Descrição
telefone	string	Sim	Telefone de contato; também é o destino de SMS e WhatsApp.
dataNascimento	Date	Sim	Data de nascimento.
canalPreferido (novo)	CanalNotificacao	Sim	Canal em que o paciente recebe as notificações.

Recepcionista (herda de Usuario): não possui atributos próprios.

Notificacao (nova)

Atributo	Tipo	Obrig.	Descrição
id	int	Sim	Identificador único.
consultaId	int (referência)	Sim	Consulta que originou a notificação.
tipo	TipoNotificacao	Sim	Confirmação, reagendamento, cancelamento ou lembrete.
canal	CanalNotificacao	Sim	Canal usado no envio, copiado do canal preferido do paciente.
destino	string	Sim	E-mail ou telefone para onde a mensagem será enviada.
mensagem	string	Sim	Texto com data, hora, médico e local ou modalidade da consulta.
dataCriacao	DateTime	Sim	Momento em que a notificação foi gerada.
status	StatusNotificacao	Sim	Pendente ou enviada.
dataEnvio	DateTime	Não	Momento em que foi entregue ao sistema de notificação.

5.3 Enumerações

Enumeração	Valores	Descrição	Situação
Modalidade	Presencial, Teleconsulta	Forma de atendimento do horário.	Sem alteração
StatusConsulta	Agendada, Cancelada, Realizada	Situação da consulta.	Sem alteração
ProvedorAutenticacao	Local, Google, Microsoft, Meta	Forma de autenticação da conta.	Nova
CanalNotificacao	Email, Sms, WhatsApp	Canais de envio de notificações.	Nova
TipoNotificacao	Confirmacao, Reagendamento, Cancelamento, Lembrete	Motivo da notificação.	Nova
StatusNotificacao	Pendente, Enviada	Situação do envio.	Nova

5.4 Associações

Associação	Classes	Multiplicidade	Descrição	Situação
herança	Paciente, Recepcionista, Medico, Administrador → Usuario	—	Os tipos de usuário herdam os dados de acesso.	Alterada
possui	Medico — Especialidade	0..* : 1	Cada médico tem uma especialidade.	Sem alteração
disponibiliza	Medico — HorarioDisponivel	1 : 0..*	Composição: o horário pertence ao médico.	Sem alteração
atende em	Medico — LocalAtendimento	1 : 0..*	Composição: locais de atendimento do médico.	Sem alteração
ocorre em	HorarioDisponivel — LocalAtendimento	0..* : 0..1	Só horários presenciais têm local.	Sem alteração
agenda	Paciente — Consulta	1 : 0..*	Consultas do paciente.	Sem alteração
ocupa	Consulta — HorarioDisponivel	0..* : 1	Horário reservado pela consulta.	Sem alteração
gera	Consulta — Notificacao	1 : 0..*	Cada agendamento, reagendamento, cancelamento ou lembrete gera uma notificação.	Nova

---

6. Diagramas de Sequência

Os diagramas a seguir mostram a troca de mensagens entre os atores e os objetos do sistema em cada caso de uso com fluxo principal descrito. Foram usados três tipos de participante: a tela, que recebe os dados do usuário; a classe de serviço, que aplica as regras do caso de uso; e as classes de entidade e repositório, que guardam os dados. Os nomes das mensagens correspondem aos métodos implementados.

Há um diagrama para cada caso de uso especificado, de UC01 a UC15. UC03 e UC04 acontecem em sequência e foram representados juntos.

UC01 — Criar conta

O cadastro só é concluído se o e-mail ainda não estiver em uso. A senha é convertida em hash pelo próprio objeto Paciente.

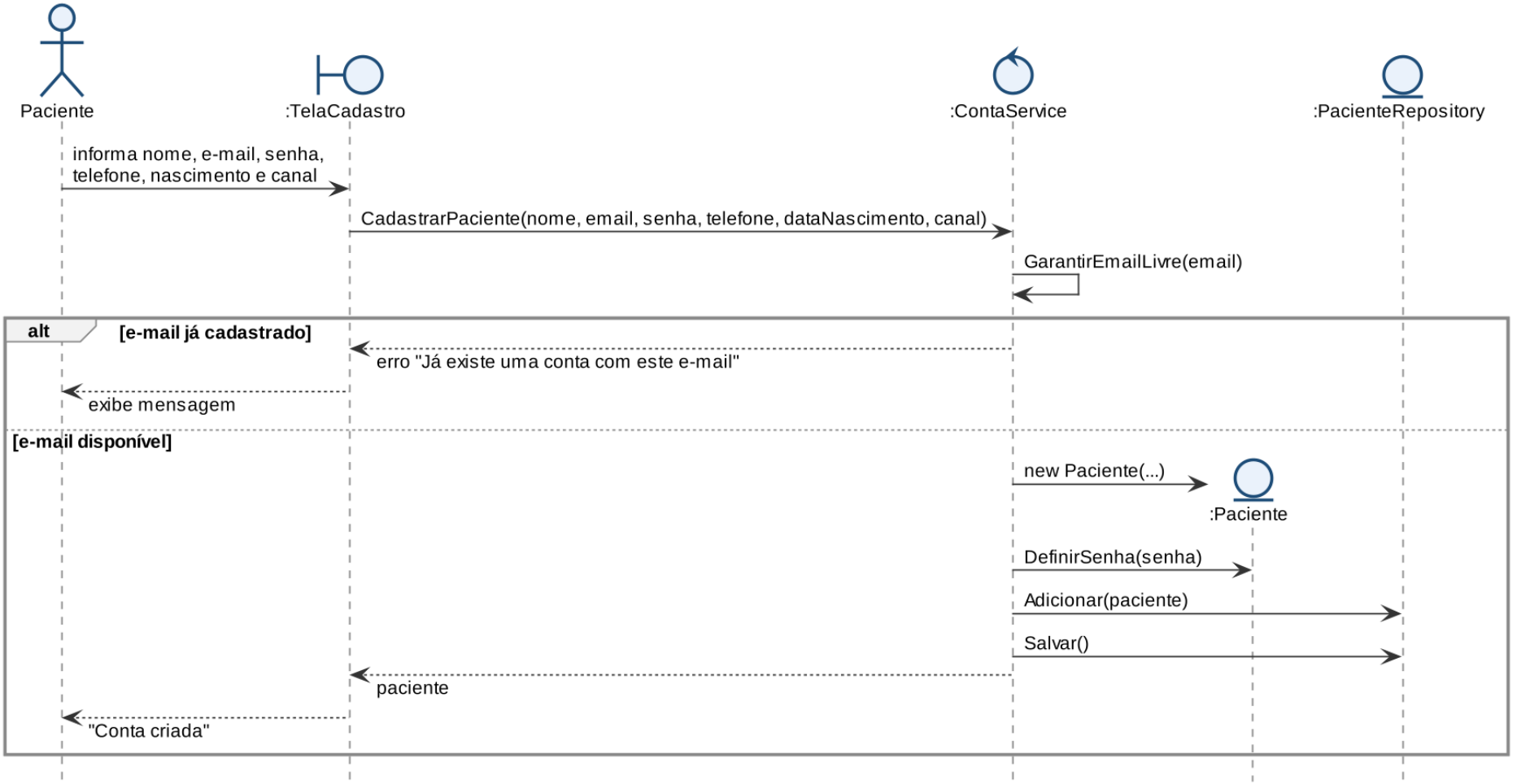


Figura 2 — UC01 — Criar conta.

UC02 — Realizar login

O fluxo alternativo com o sistema de autenticação SSO está previsto no modelo; nesta etapa o sistema identifica contas externas e orienta o usuário.

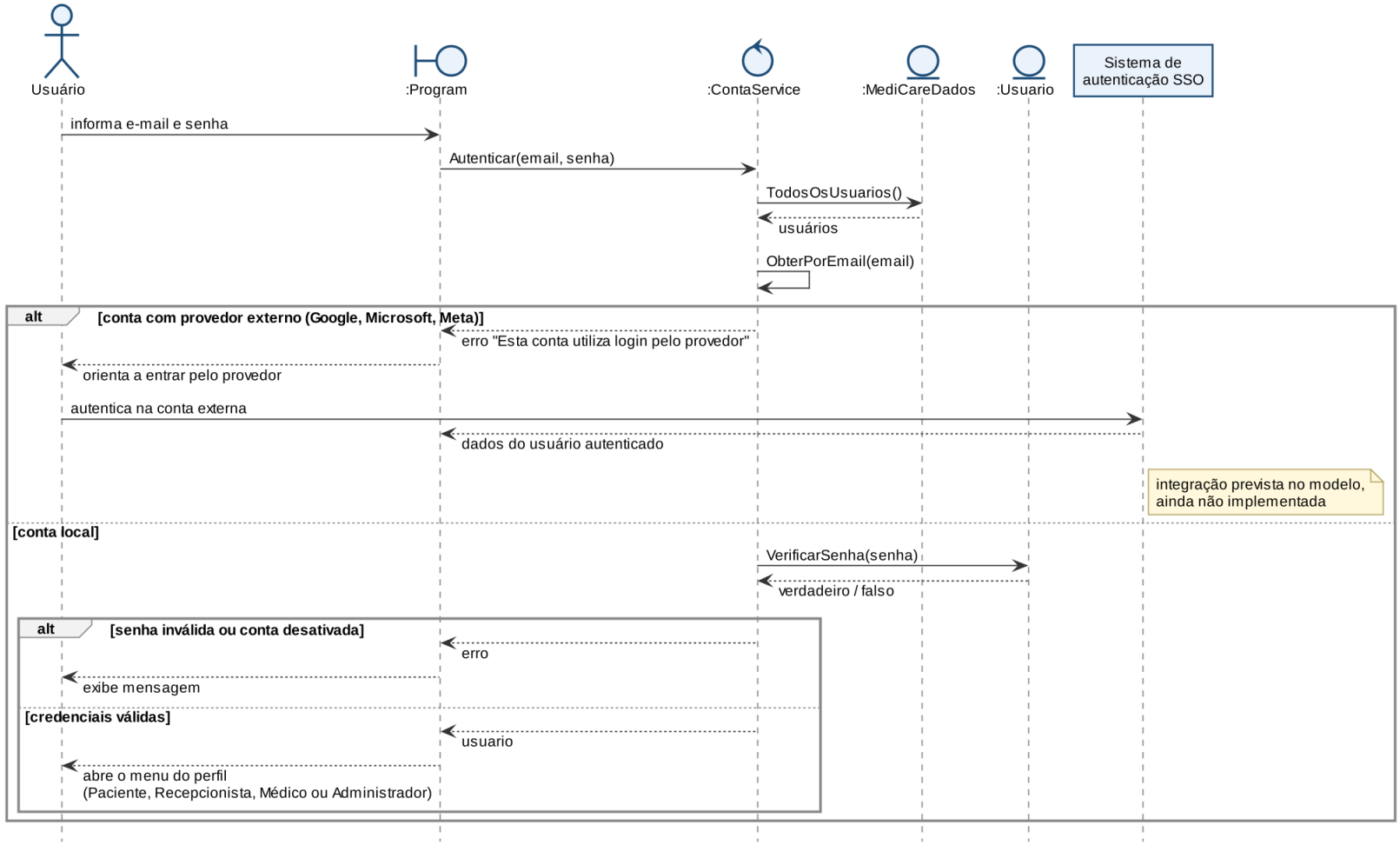


Figura 3 — UC02 — Realizar login.

UC03 e UC04 — Pesquisar médicos e consultar horários

Os dois casos de uso acontecem em sequência e usam o mesmo serviço.

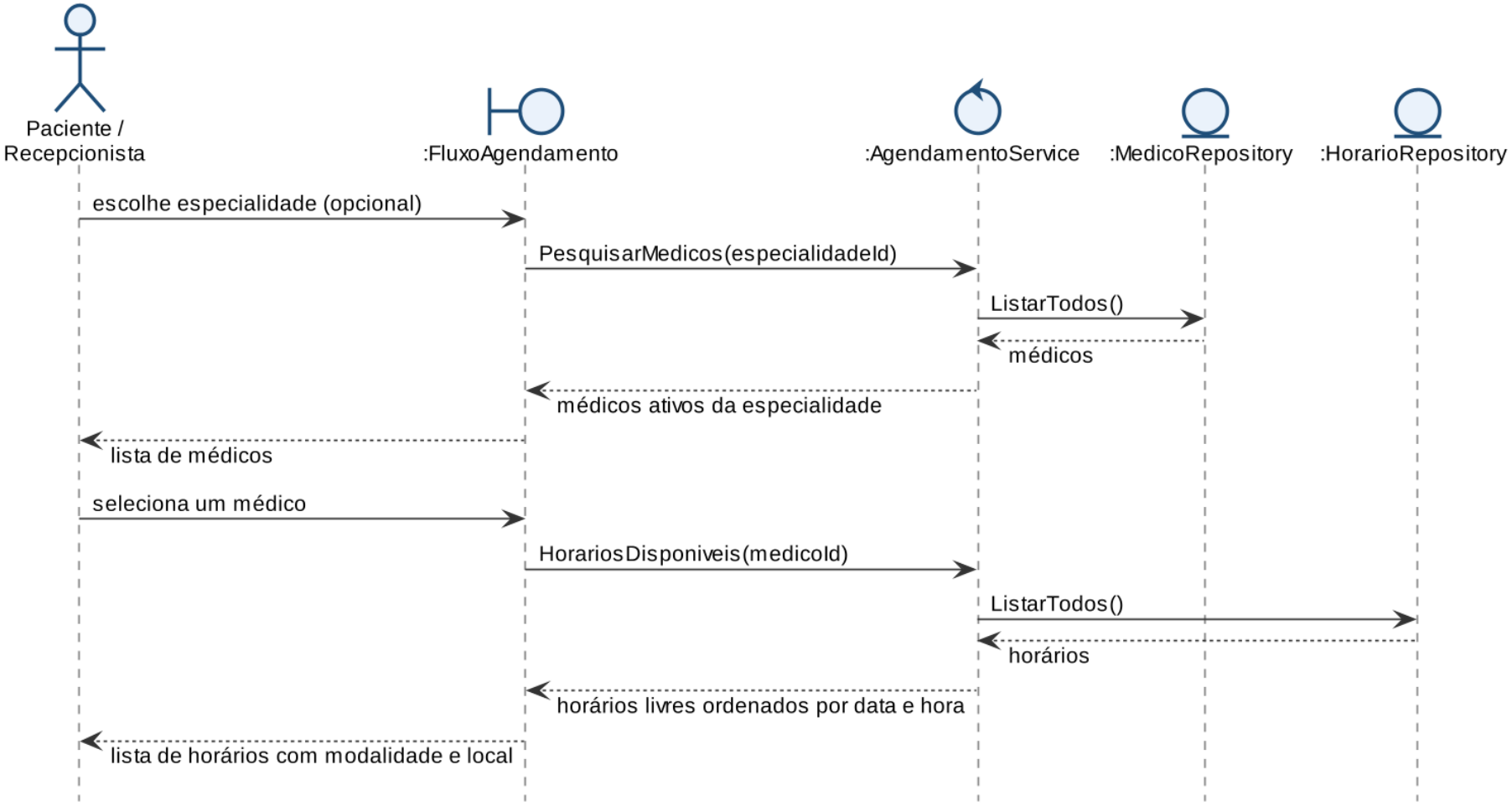


Figura 4 — UC03 e UC04 — Pesquisar médicos e consultar horários.

UC05 — Agendar consulta

A reserva do horário é feita antes de criar a consulta. Se o horário já estiver reservado, nada é gravado.

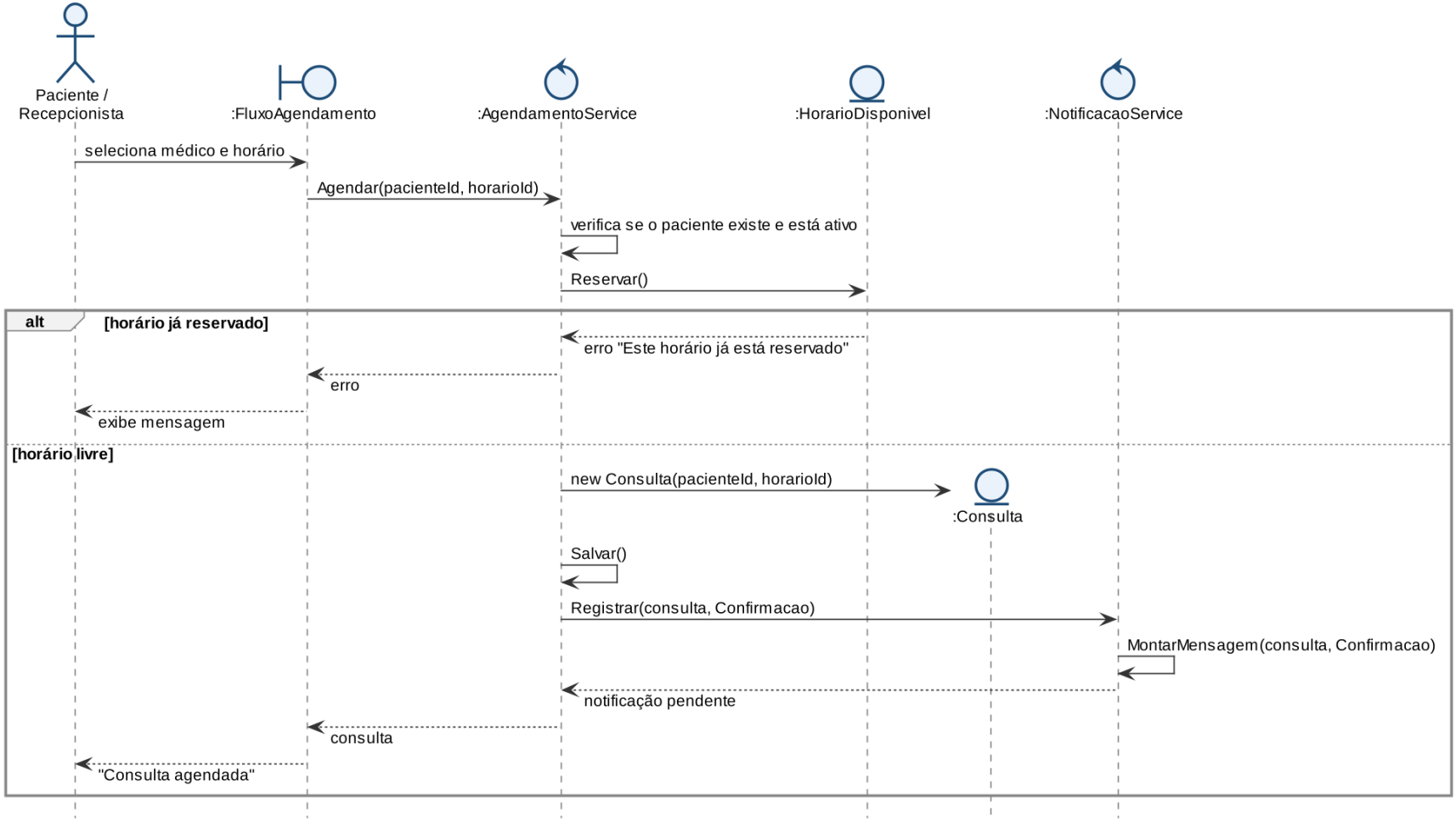


Figura 5 — UC05 — Agendar consulta.

UC06 — Cancelar consulta

A tela só permite escolher entre as consultas agendadas do próprio paciente.

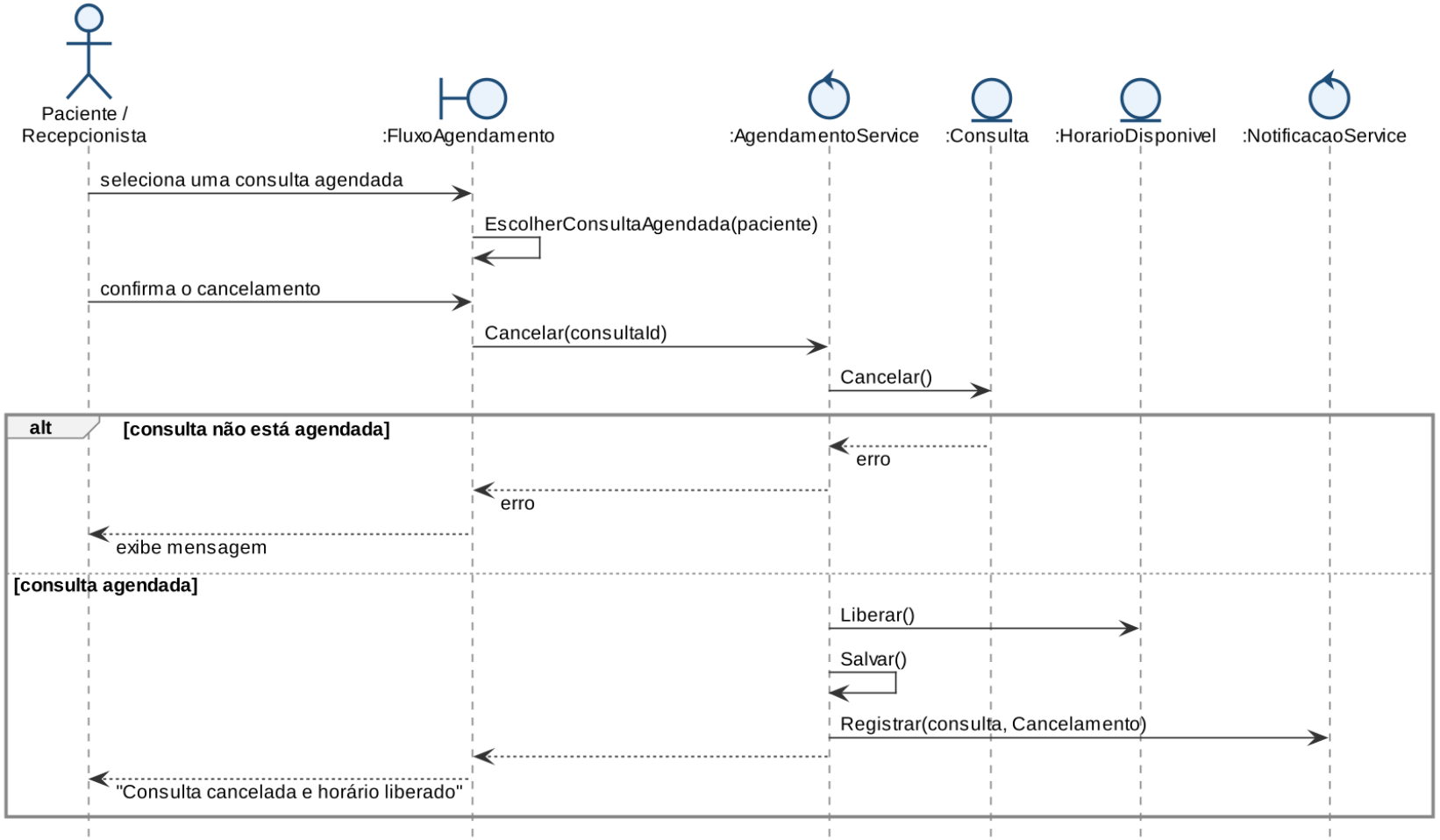


Figura 6 — UC06 — Cancelar consulta.

UC07 — Reagendar consulta

O novo horário é validado antes de qualquer alteração, para que a consulta não fique em estado inconsistente.

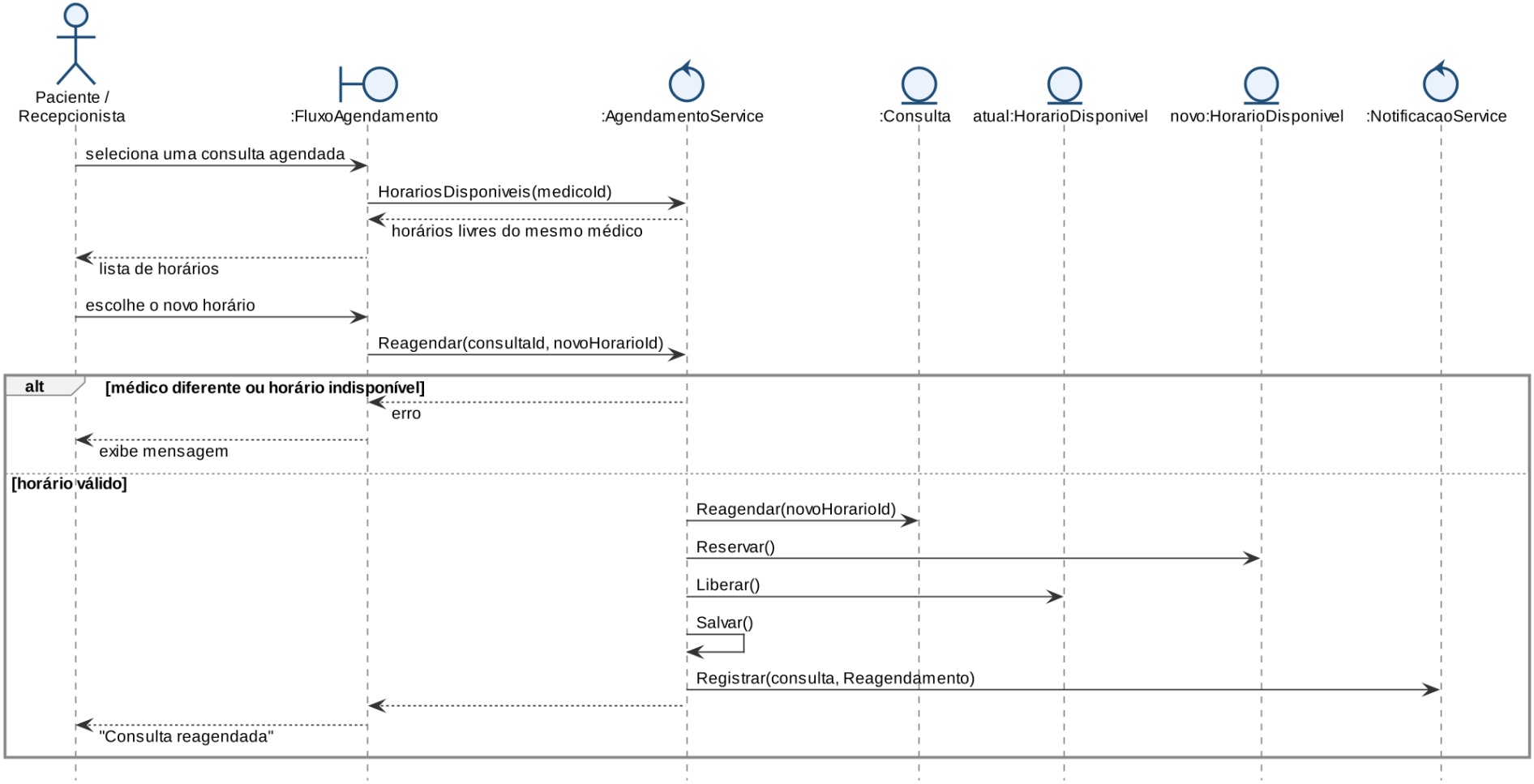


Figura 7 — UC07 — Reagendar consulta.

UC08 — Gerenciar perfil

O paciente atualiza telefone e canal de notificação; o médico atualiza o telefone pelo método equivalente.

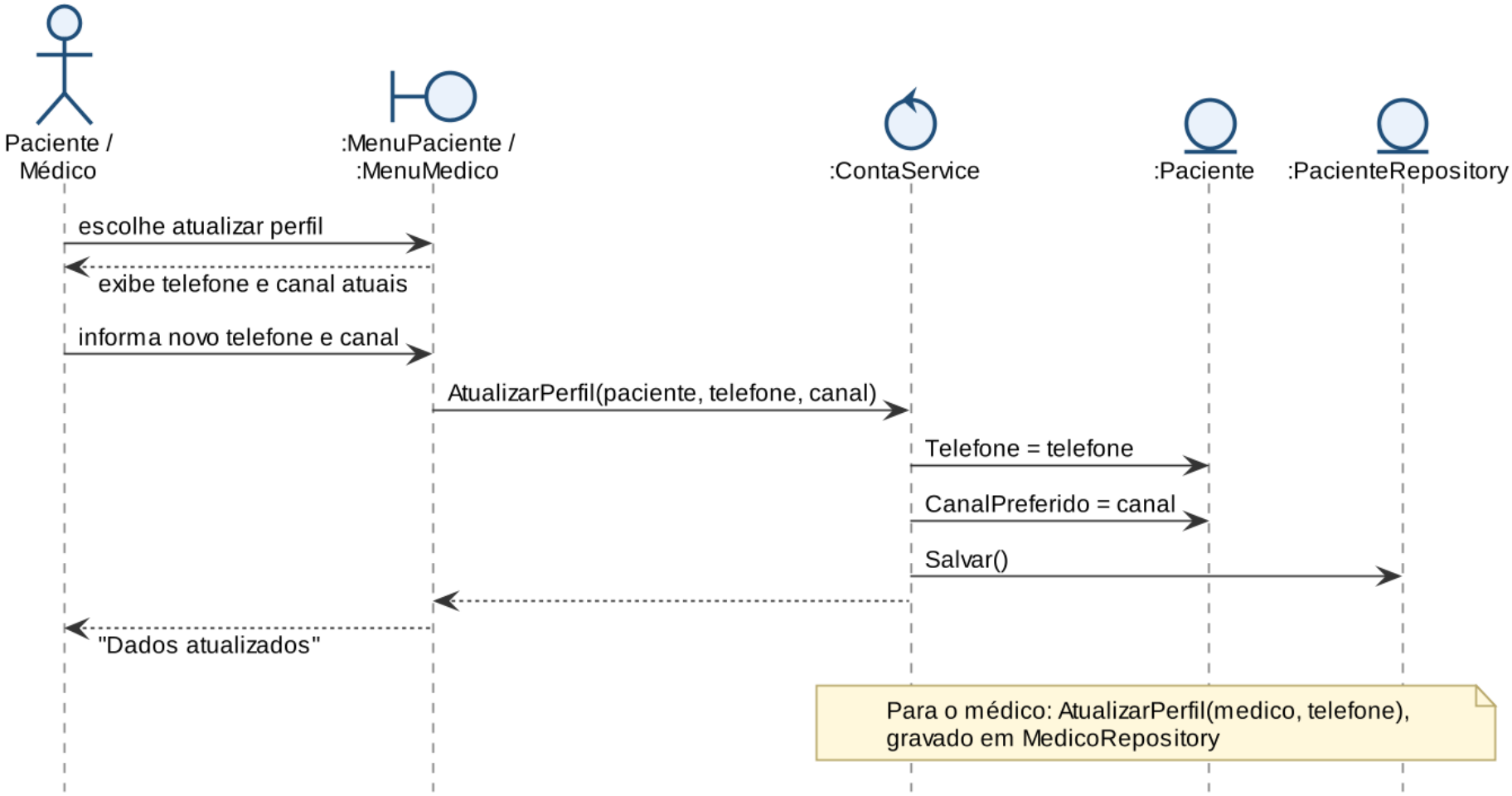


Figura 8 — UC08 — Gerenciar perfil.

UC09 — Gerenciar disponibilidade

Cada novo horário é comparado com os horários já cadastrados do médico pelo método ConflitaCom().

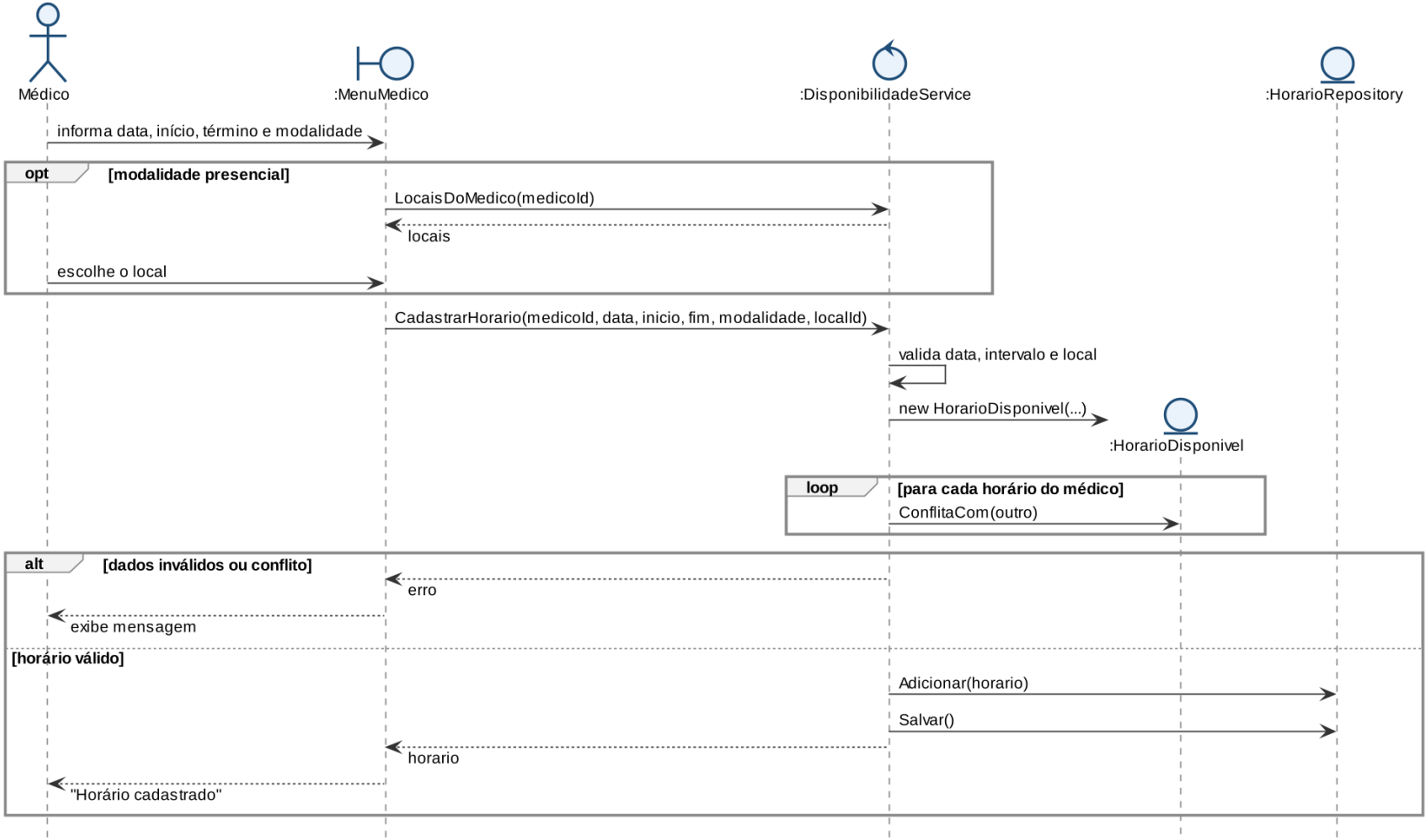


Figura 9 — UC09 — Gerenciar disponibilidade.

UC10 — Gerenciar agenda

Registrar a consulta como realizada é o que permite que ela apareça no histórico do paciente (UC15).

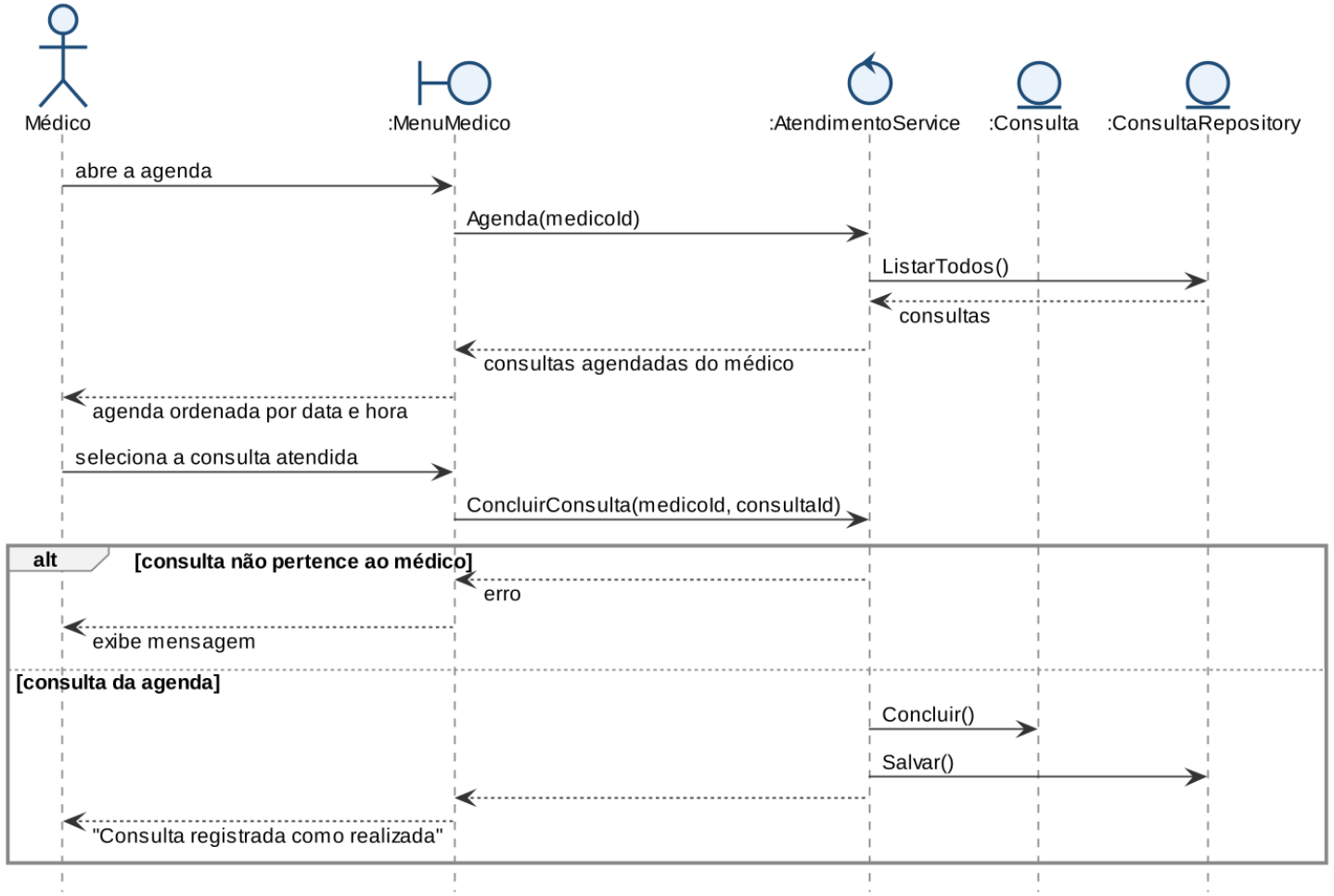


Figura 10 — UC10 — Gerenciar agenda.

UC11 — Realizar teleconsulta

O acesso é liberado somente ao paciente e ao médico da consulta, e somente dentro do horário agendado. A chamada de vídeo é feita pela plataforma de teleconsulta.

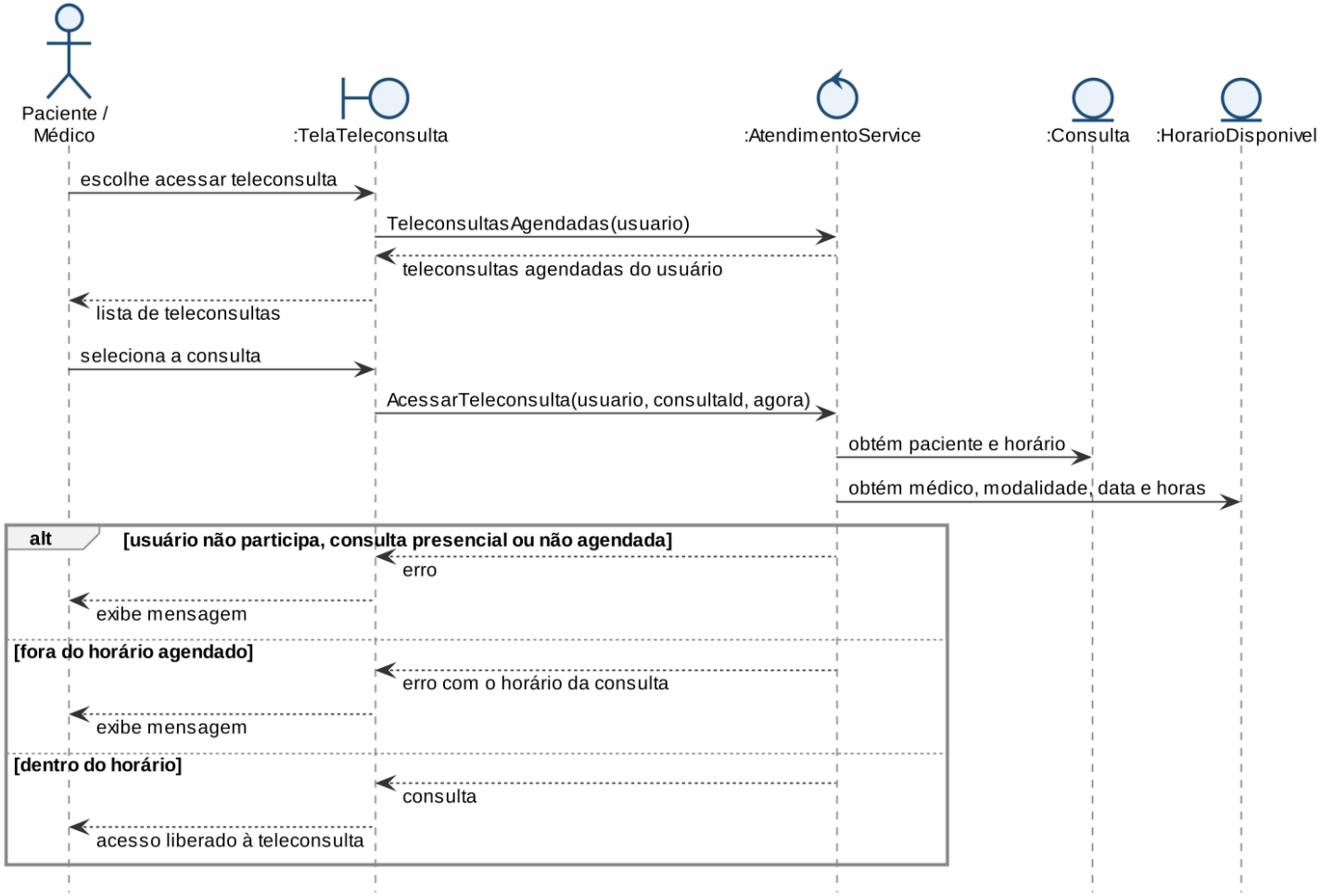


Figura 11 — UC11 — Realizar teleconsulta.

UC12 — Gerenciar usuários

A busca por e-mail percorre todos os tipos de usuário, e a gravação atualiza os arquivos de usuários.

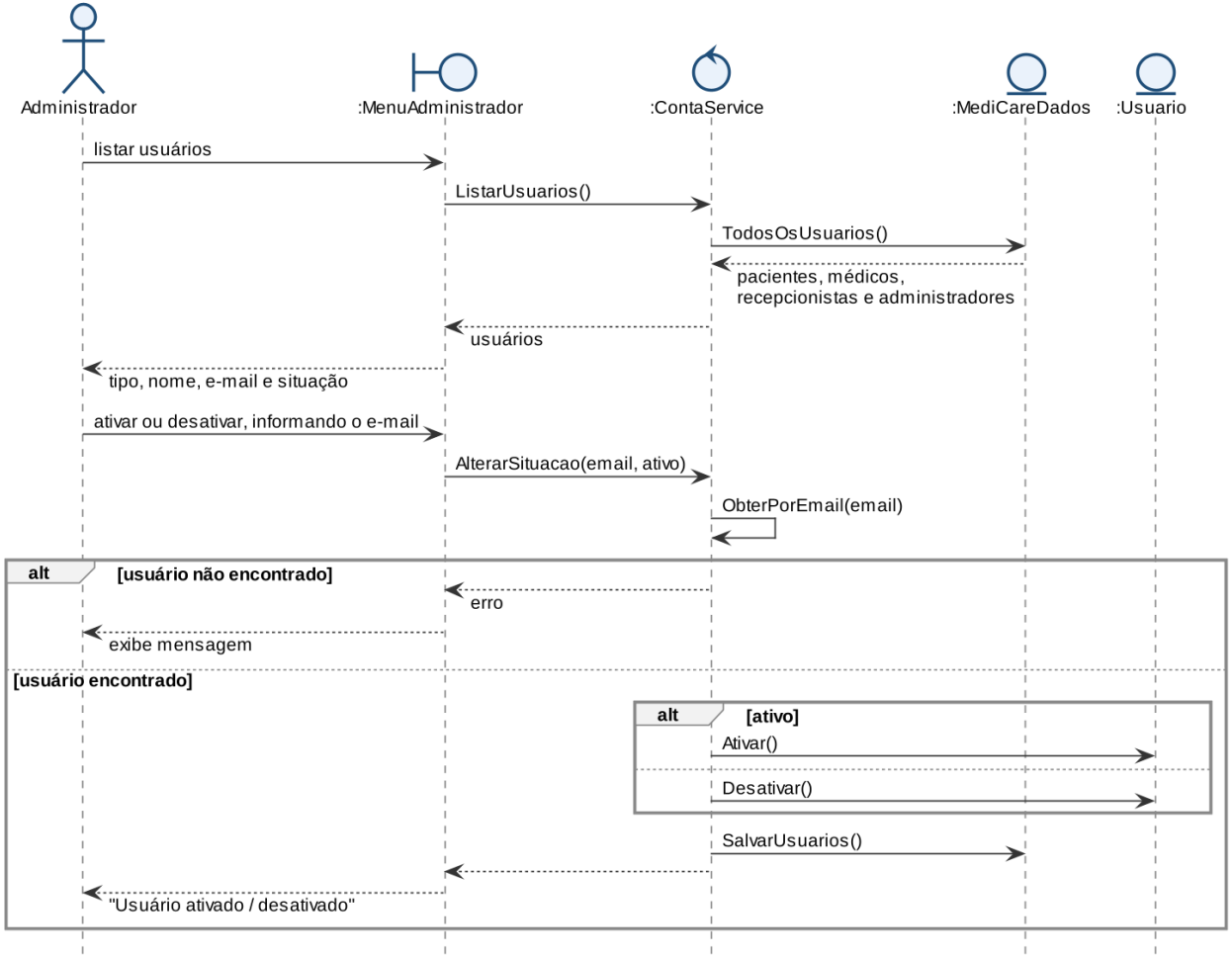


Figura 12 — UC12 — Gerenciar usuários.

UC13 — Gerenciar agendamentos

O cancelamento feito pelo administrador reaproveita o fluxo de UC06.

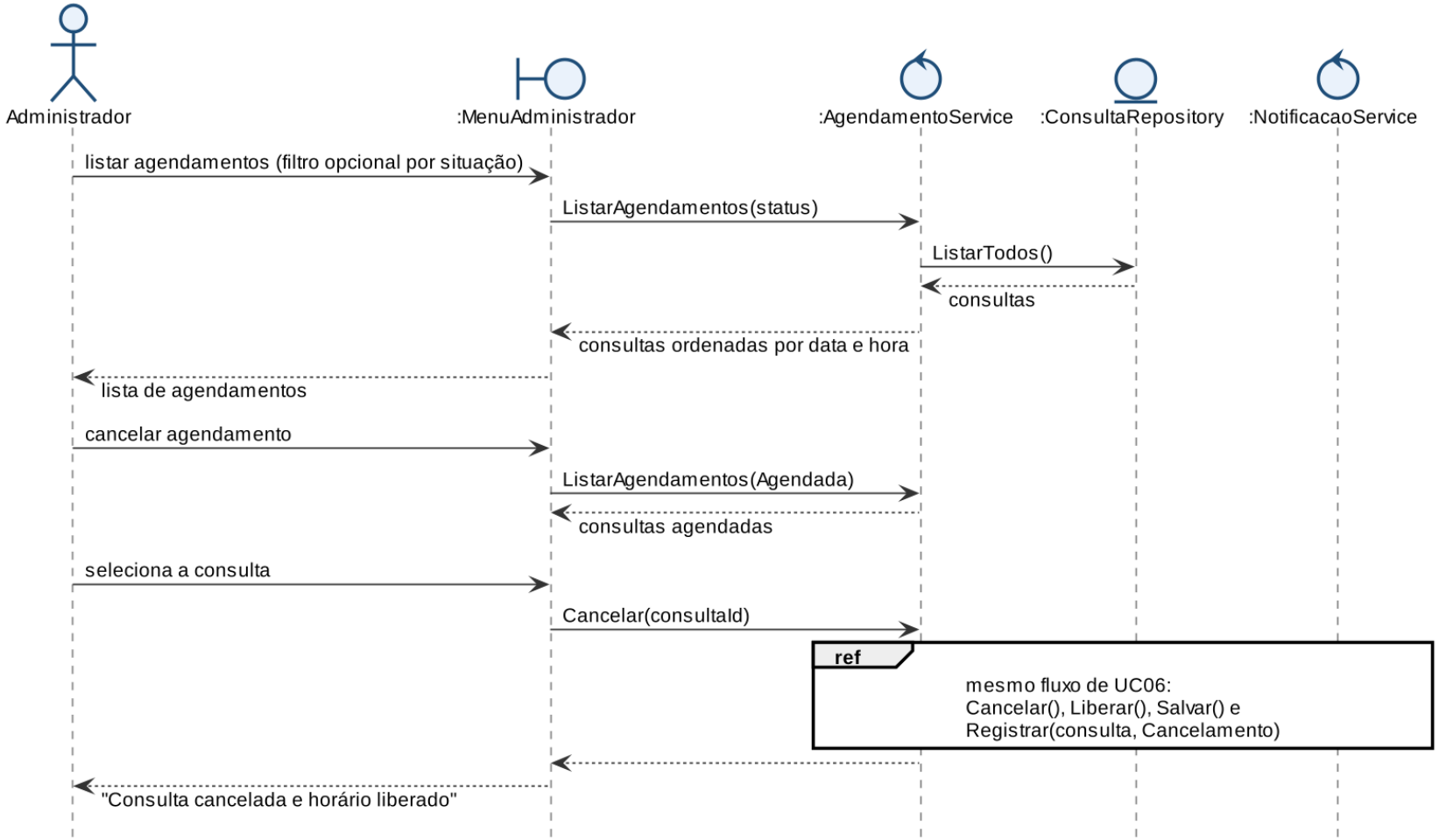


Figura 13 — UC13 — Gerenciar agendamentos.

UC14 — Enviar notificações

As notificações de confirmação, reagendamento e cancelamento são geradas nos UC05 a UC07; aqui são gerados os lembretes e feita a entrega ao sistema de notificação.

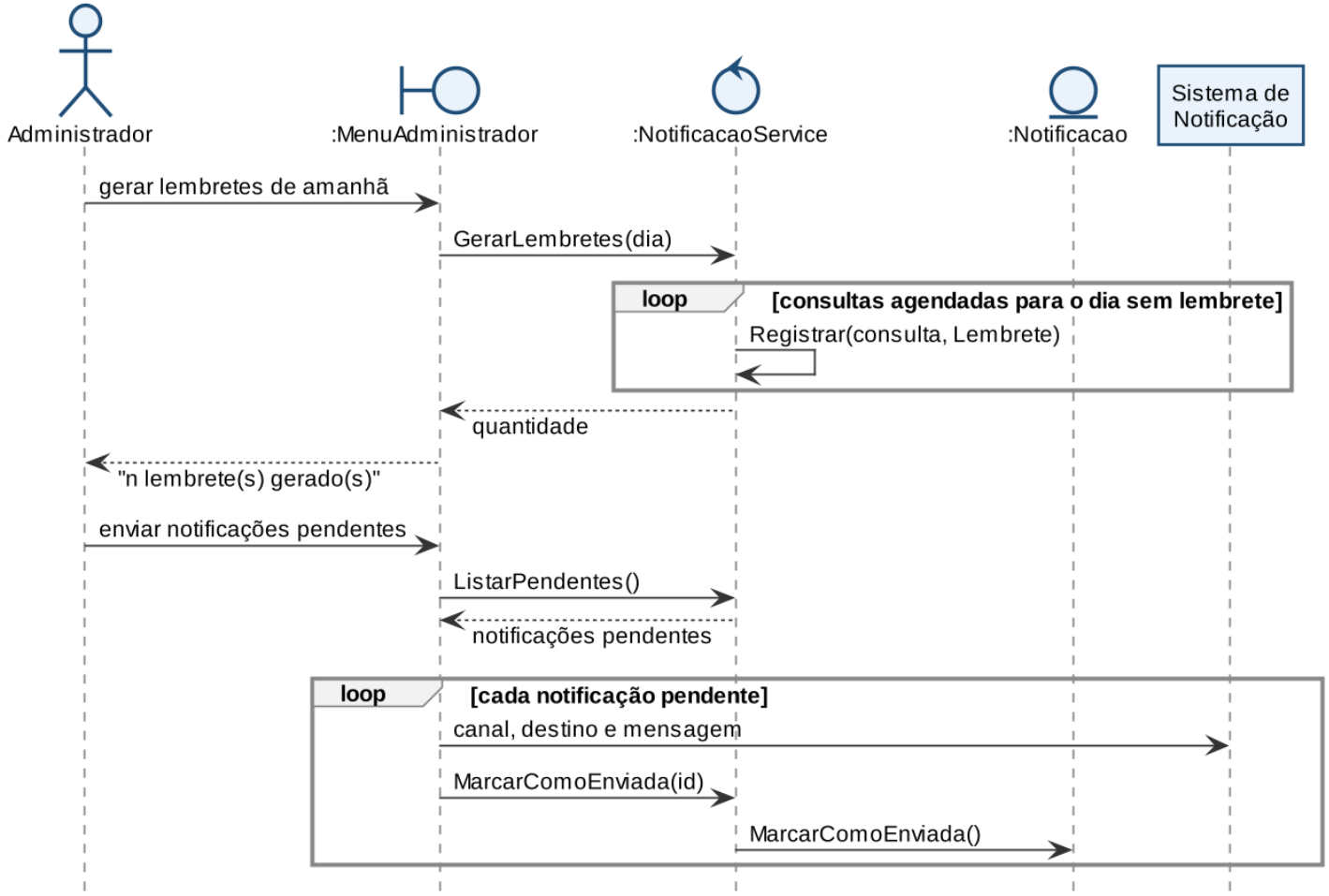


Figura 14 — UC14 — Enviar notificações.

UC15 — Consultar histórico do paciente

O médico só vê o histórico de pacientes que tenham ao menos uma consulta com ele.

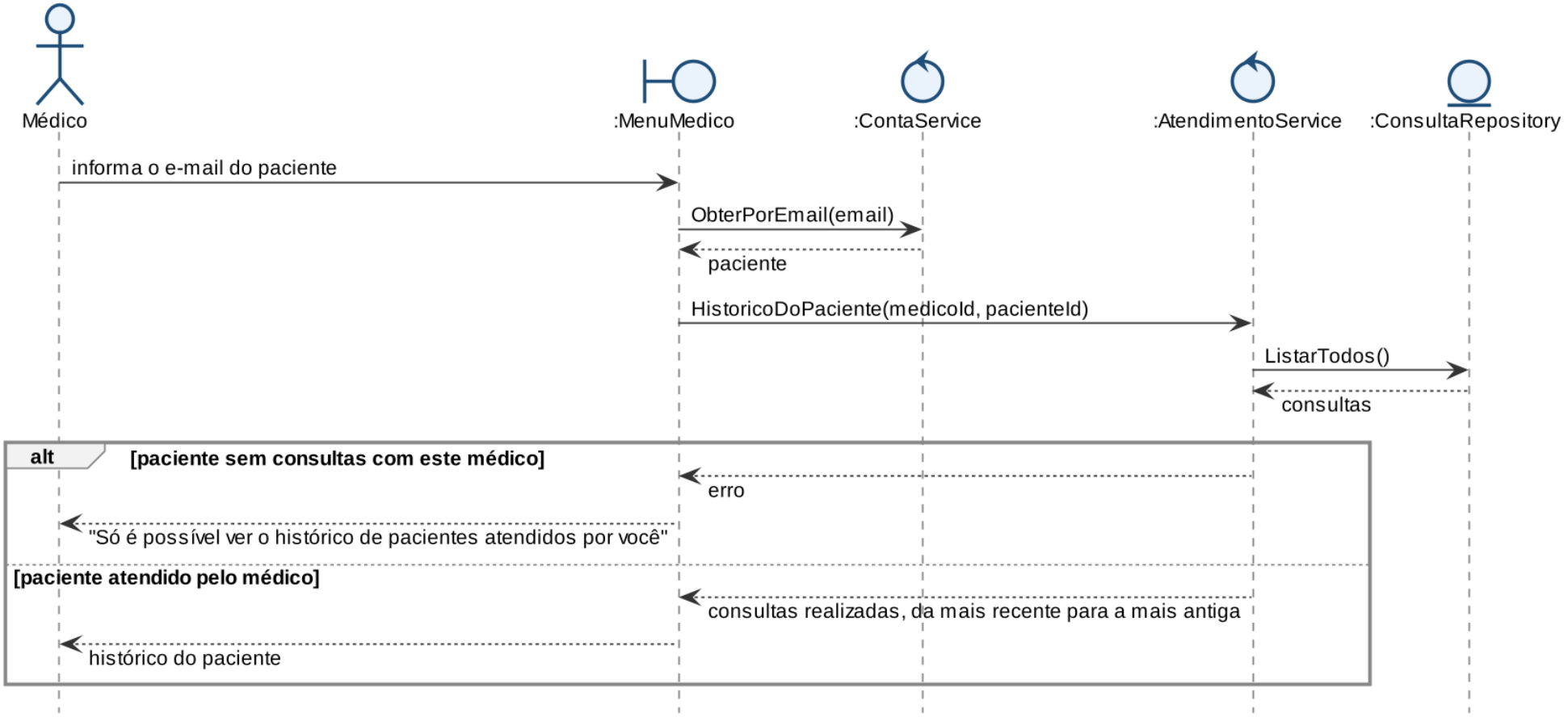


Figura 15 — UC15 — Consultar histórico do paciente.

7. Diagramas de Comunicação

Os diagramas de comunicação mostram as mesmas interações dos diagramas de sequência, com foco nas ligações entre os objetos. Cada linha representa uma ligação, e as mensagens trocadas por ela aparecem numeradas na ordem em que acontecem. A seta “→” indica que a mensagem vai do objeto de cima para o de baixo; “←” indica o retorno. O asterisco (*) marca mensagens repetidas para cada item de uma lista.

Foram feitos diagramas de comunicação para os casos de uso com mais objetos envolvidos ou com regras de negócio próprias.

UC02 — Realizar login

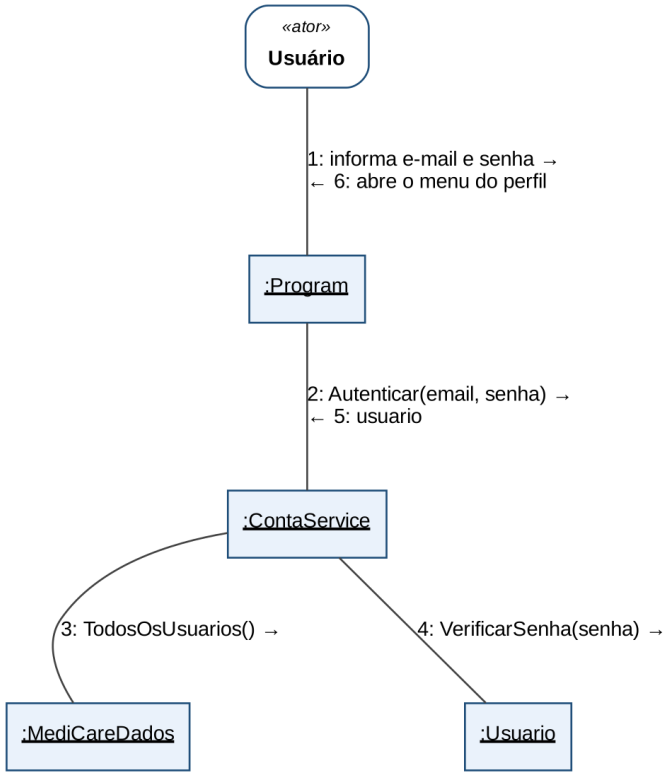


Figura 16 — Diagrama de comunicação de UC02 — Realizar login.

UC05 — Agendar consulta

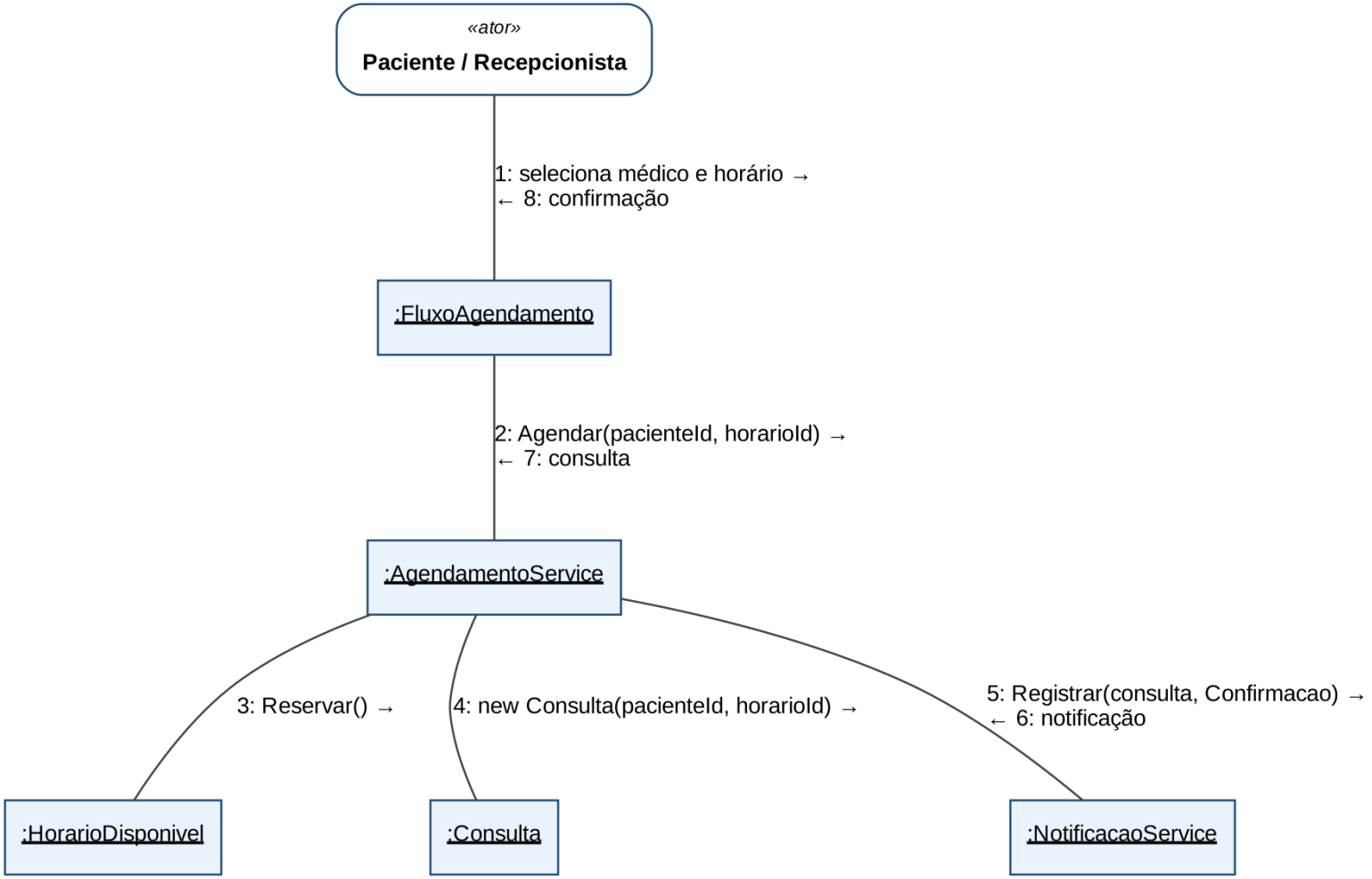


Figura 17 — Diagrama de comunicação de UC05 — Agendar consulta.

UC06 — Cancelar consulta

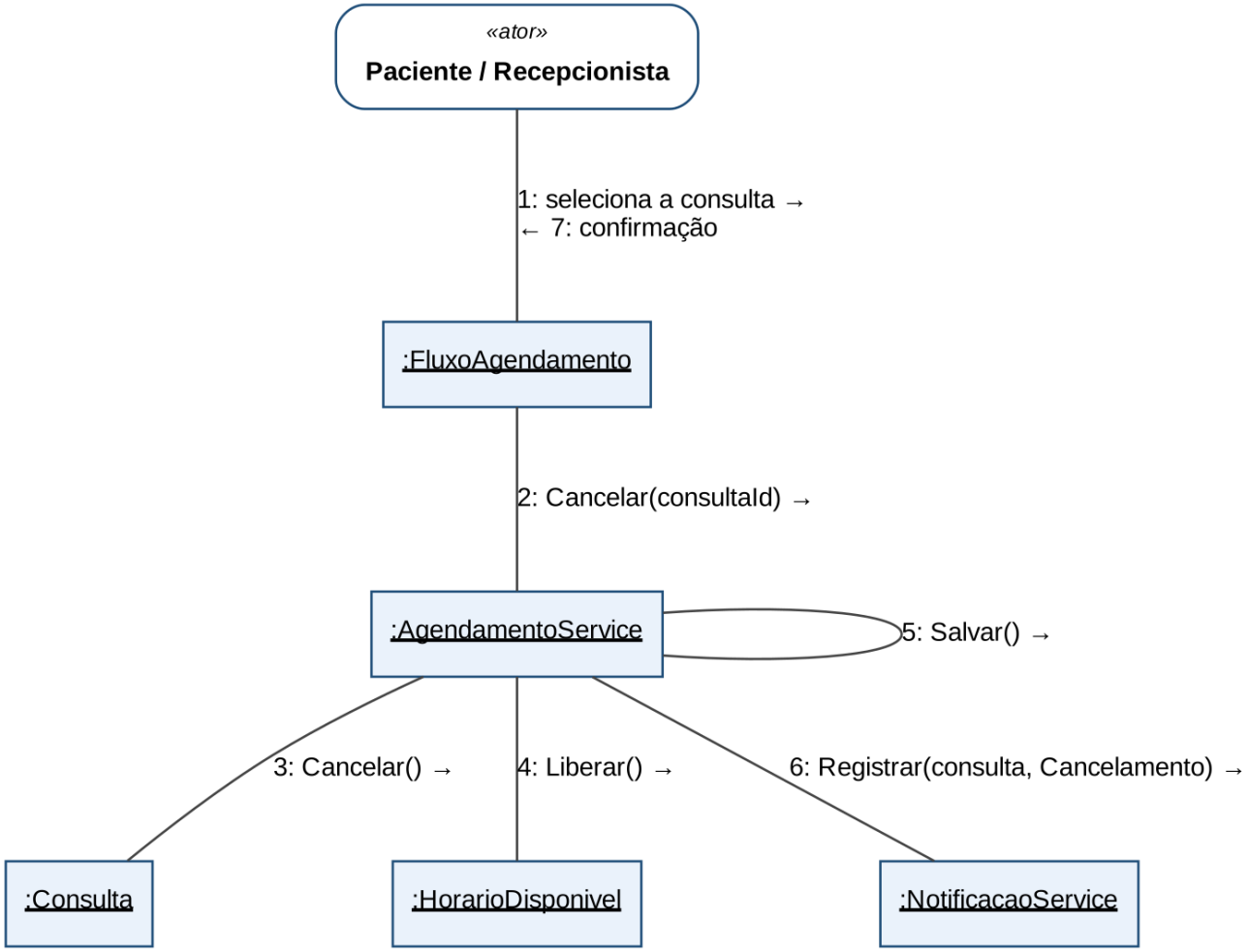


Figura 18 — Diagrama de comunicação de UC06 — Cancelar consulta.

UC07 — Reagendar consulta

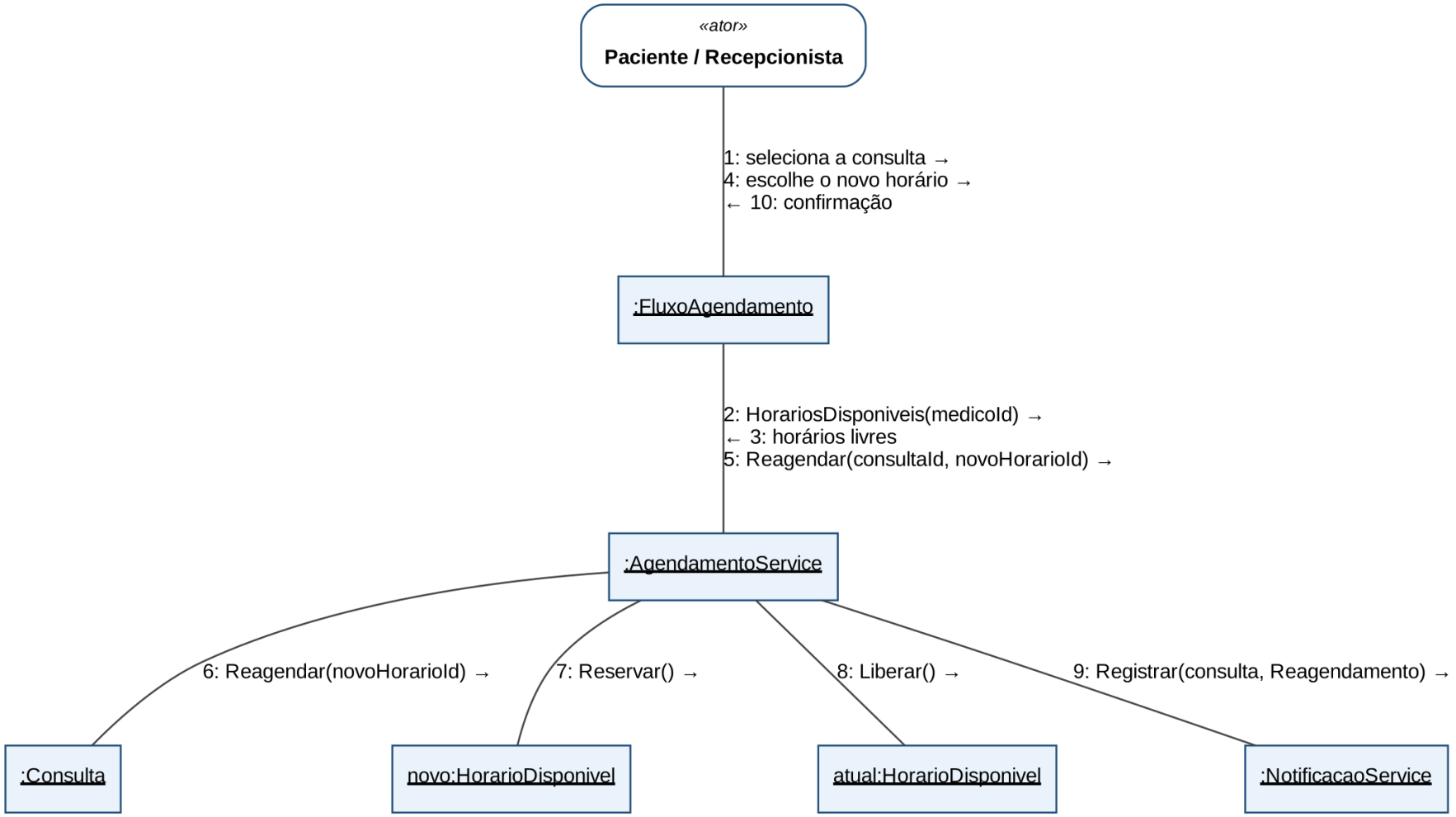


Figura 19 — Diagrama de comunicação de UC07 — Reagendar consulta.

UC09 — Gerenciar disponibilidade

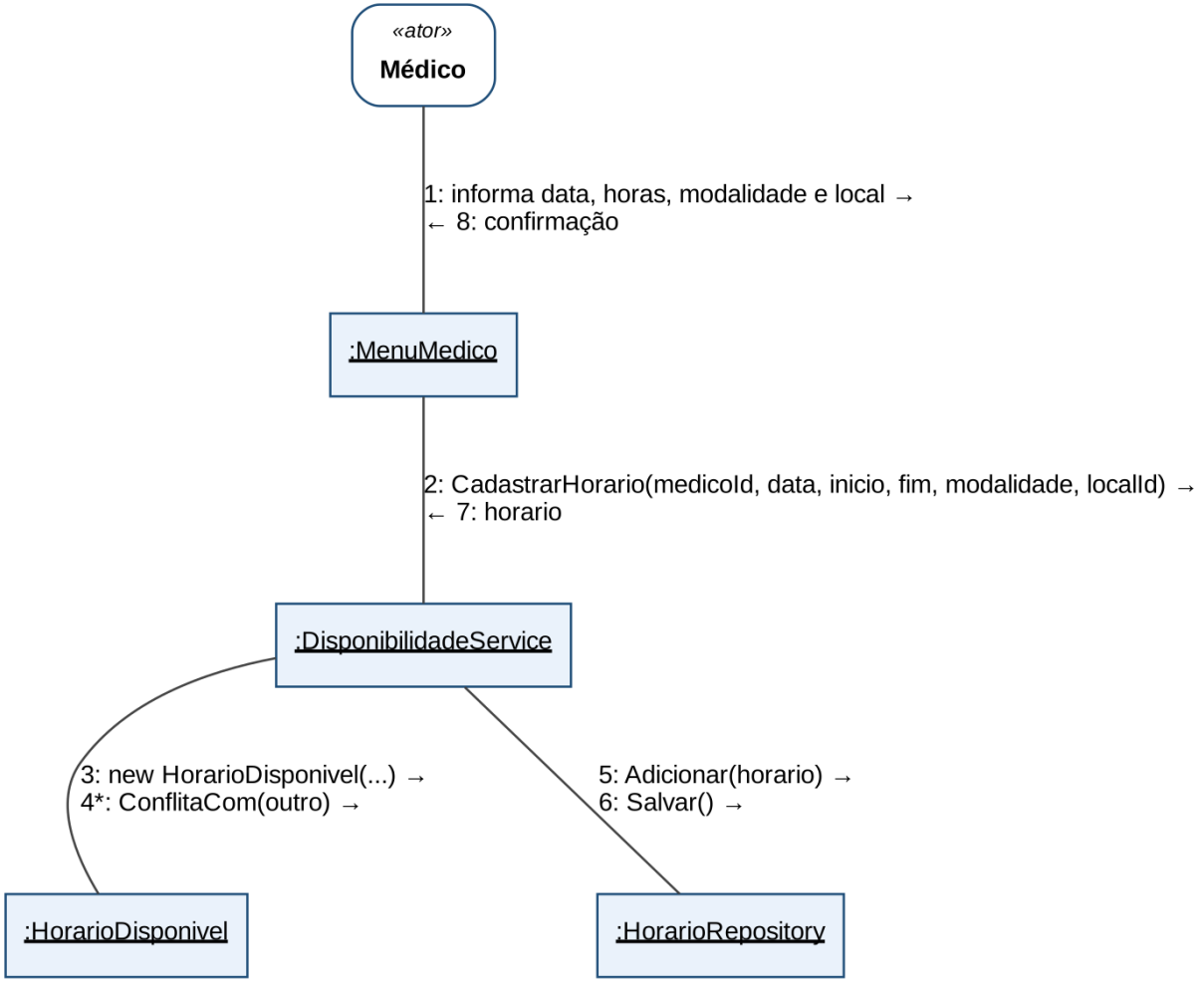


Figura 20 — Diagrama de comunicação de UC09 — Gerenciar disponibilidade.

UC11 — Realizar teleconsulta

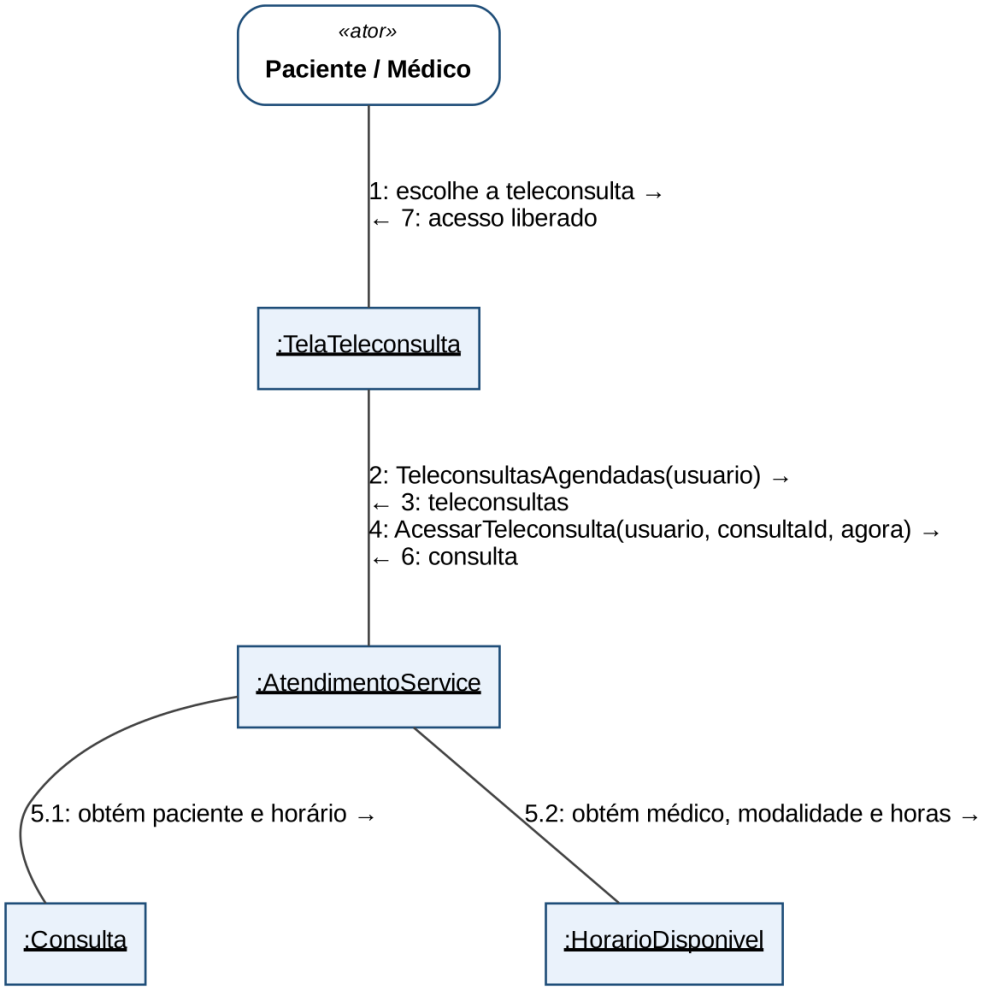


Figura 21 — Diagrama de comunicação de UC11 — Realizar teleconsulta.

UC14 — Enviar notificações

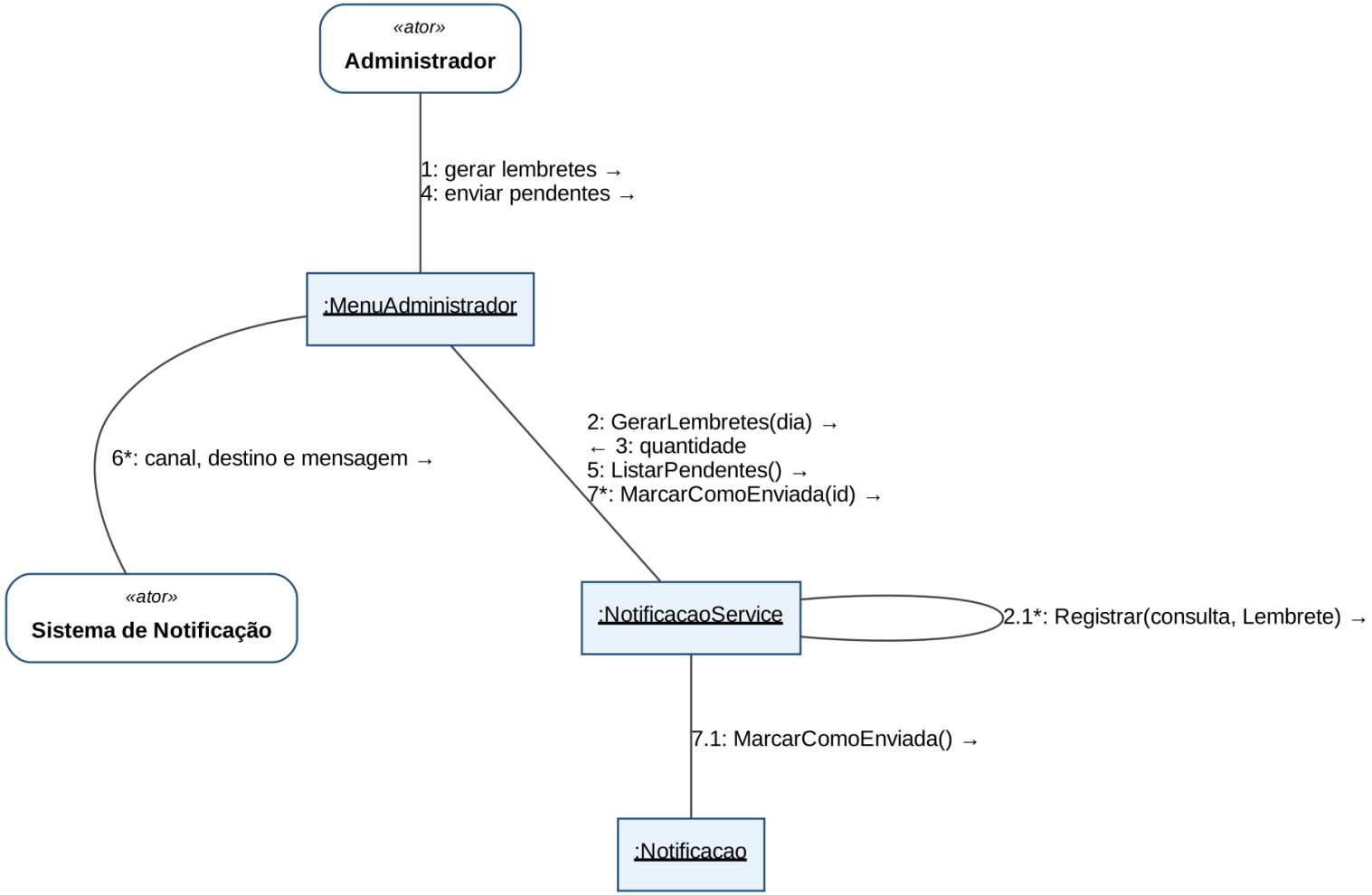


Figura 22 — Diagrama de comunicação de UC14 — Enviar notificações.

UC15 — Consultar histórico do paciente

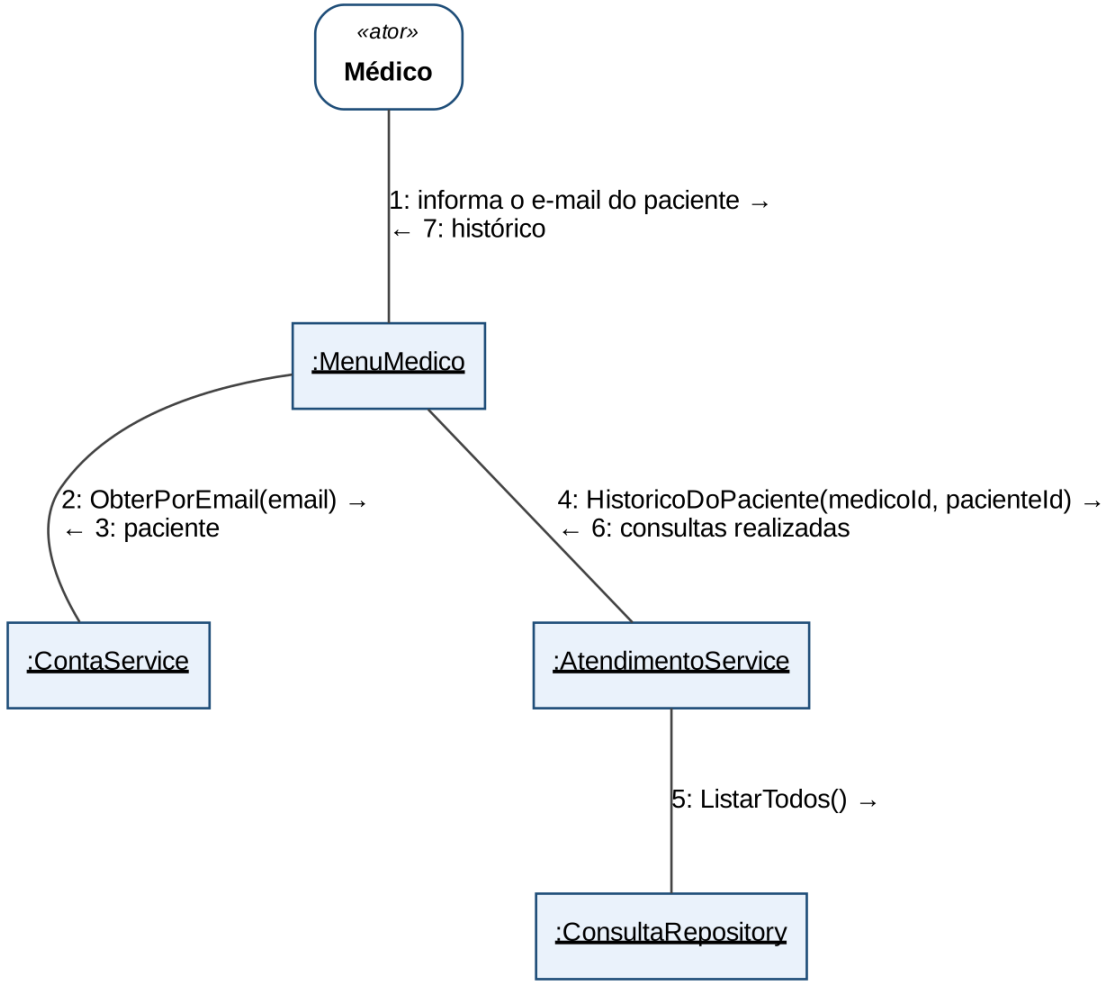


Figura 23 — Diagrama de comunicação de UC15 — Consultar histórico do paciente.

8. Projeto: Métodos das Classes

Com base nos diagramas de sequência e de comunicação, foram definidos os métodos de cada classe. Além das classes de entidade do diagrama de classes, o projeto passa a ter **classes de serviço**, que concentram as regras de cada grupo de casos de uso e coordenam as entidades e os repositórios.

8.1 Novas classes, atributos e relações

Elemento	Tipo	Motivo
Recepcionista	Classe de entidade	Mudança M1.
Notificacao	Classe de entidade	Mudança M2.
Usuario.provedor, Paciente.canalPreferido	Atributos	Mudanças M4 e M2.
Consulta — Notificacao (gera)	Associação	Mudança M2.
ContaService	Classe de serviço	UC01, UC02, UC08 e UC12.
AgendamentoService	Classe de serviço	UC03 a UC07 e UC13 (já existia; passou a gerar notificações).
DisponibilidadeService	Classe de serviço	UC09.
AtendimentoService	Classe de serviço	UC10, UC11 e UC15.
NotificacaoService	Classe de serviço	UC14 e notificações de UC05 a UC07.
UsuarioRepository<T>	Classe de persistência	Base comum para os arquivos de usuários.
RecepcionistaRepository, NotificacaoRepository	Classes de persistência	Arquivos recepcionistas.csv e notificacoes.csv.

8.2 Classes de entidade

Usuario

Cabeçalho do método	Finalidade
`void DefinirSenha(string senha)`	Gera e guarda o hash da senha.
`bool VerificarSenha(string senha)`	Compara a senha informada; sempre falso para contas externas.
`void Ativar()`	Reativa a conta (UC12).
`void Desativar()`	Impede o acesso da conta (UC12).

HorarioDisponivel

Cabeçalho do método	Finalidade
`void Reservar()`	Marca o horário como ocupado; erro se já estiver reservado.
`void Liberar()`	Torna o horário disponível novamente.
`bool ConflitaCom(HorarioDisponivel outro)`	Verifica sobreposição com outro horário do mesmo médico no mesmo dia.

Consulta

Cancelar, Reagendar e Concluir só são aceitos quando a consulta está agendada.

Cabeçalho do método	Finalidade
`Consulta(int pacienteId, int horarioId)`	Cria a consulta com status Agendada.
`void Cancelar()`	Altera o status para Cancelada.
`void Reagendar(int novoHorarioId)`	Troca o horário da consulta.
`void Concluir()`	Altera o status para Realizada.

Notificacao

Cabeçalho do método	Finalidade
`Notificacao(int consultaId, TipoNotificacao tipo, CanalNotificacao canal, string destino, string mensagem)`	Cria a notificação com status Pendente.
`void MarcarComoEnviada()`	Registra o envio e a data; erro se já tiver sido enviada.

Paciente, Medico, Recepcionista, Administrador, Especialidade e LocalAtendimento possuem apenas atributos, sem métodos próprios.

8.3 Classes de serviço

ContaService

Cabeçalho do método	Finalidade
`Paciente CadastrarPaciente(string nome, string email, string senha, string telefone, DateOnly dataNascimento, CanalNotificacao canal)`	UC01 para pacientes.
`Medico CadastrarMedico(string nome, string email, string senha, string crm, string telefone, int especialidadeId)`	UC01 para médicos.
`Usuario Autenticar(string email, string senha)`	UC02.
`void AtualizarPerfil(Paciente paciente, string telefone, CanalNotificacao canal)`	UC08 — paciente.
`void AtualizarPerfil(Medico medico, string telefone)`	UC08 — médico.
`Usuario? ObterPorEmail(string email)`	Localiza um usuário de qualquer tipo.
`IEnumerable<Usuario> ListarUsuarios()`	UC12.
`void AlterarSituacao(string email, bool ativo)`	UC12.

AgendamentoService

Cabeçalho do método	Finalidade
`IEnumerable<Medico> PesquisarMedicos(int? especialidadeId)`	UC03.
`IEnumerable<HorarioDisponivel> HorariosDisponiveis(int medicoId)`	UC04.
`IEnumerable<Consulta> ConsultasDoPaciente(int pacienteId)`	Consultas de um paciente.
`Consulta Agendar(int pacienteId, int horarioId)`	UC05.
`void Cancelar(int consultaId)`	UC06.
`void Reagendar(int consultaId, int novoHorarioId)`	UC07.
`IEnumerable<Consulta> ListarAgendamentos(StatusConsulta? status)`	UC13 — lista com filtro opcional.

DisponibilidadeService

Cabeçalho do método	Finalidade
`IEnumerable<LocalAtendimento> LocaisDoMedico(int medicoId)`	Locais para horários presenciais.
`IEnumerable<HorarioDisponivel> HorariosDoMedico(int medicoId)`	Todos os horários do médico.
`HorarioDisponivel CadastrarHorario(int medicoId, DateOnly data, TimeOnly inicio, TimeOnly fim, Modalidade modalidade, int? localId)`	UC09 — cadastro com validações.
`void RemoverHorario(int medicoId, int horarioId)`	UC09 — remoção de horário não utilizado.

AtendimentoService

Cabeçalho do método	Finalidade
`IEnumerable<Consulta> Agenda(int medicoId)`	UC10.
`void ConcluirConsulta(int medicoId, int consultaId)`	UC10 — registra atendimento.
`IEnumerable<Consulta> HistoricoDoPaciente(int medicoId, int pacienteId)`	UC15.
`IEnumerable<Consulta> TeleconsultasAgendadas(Usuario usuario)`	UC11 — teleconsultas do usuário.
`Consulta AcessarTeleconsulta(Usuario usuario, int consultaId, DateTime agora)`	UC11 — libera o acesso.

NotificacaoService

Cabeçalho do método	Finalidade
`Notificacao Registrar(Consulta consulta, TipoNotificacao tipo)`	Cria a notificação no canal do paciente.
`int GerarLembretes(DateOnly dia)`	UC14 — lembretes das consultas do dia.
`IEnumerable<Notificacao> ListarPendentes()`	UC14 — notificações a enviar.
`void MarcarComoEnviada(int notificacaoId)`	UC14 — confirma a entrega.

8.4 Classes de persistência

Todos os repositórios herdam de **CsvRepository<T>**, que oferece *ListarTodos()*, *ObterPorId(int id)*, *Adicionar(T item)*, *Remover(int id)* e *Salvar()*. Os repositórios de usuários herdam de **UsuarioRepository<T>**, que trata as colunas comuns a todos os usuários; cada repositório filho define apenas as colunas próprias do seu tipo. A classe **MediCareDados** reúne os repositórios e oferece *TodosOsUsuarios()* e *SalvarUsuarios()*.

---

9. Implementação

9.1 Revisão das partes já implementadas

- O modelo recebeu as classes Recepcionista e Notificacao, os novos atributos e as novas enumerações.
- Os repositórios de usuários passaram a compartilhar a classe base UsuarioRepository<T>, o que eliminou a repetição das colunas comuns em quatro classes.
- A regra de cadastro de paciente, que ficava no programa de console, foi movida para ContaService.
- O programa de console foi reorganizado: o usuário faz login e recebe o menu do seu perfil. Cada perfil tem sua própria tela, e o fluxo de agendamento é compartilhado entre paciente e recepcionista.

- Foram implementados os métodos de UC08 (perfil do paciente e do médico), UC11 (acesso à teleconsulta no horário agendado) e UC13 (listagem com filtro e cancelamento pelo administrador).
- O método SalvarTudo(), que não era usado pelo sistema, foi removido. Cada serviço grava apenas os arquivos que altera.
- Os arquivos CSV de usuários receberam a coluna Provedor, e o de pacientes a coluna CanalPreferido. Foram criados recepcionistas.csv e notificacoes.csv.

9.2 Estrutura do projeto

```
MediCare/
├── MediCare.sln
├── dados/                              9 arquivos CSV
├── src/MediCare.Core/
│   ├── Models/                         entidades e enumerações
│   ├── Csv/                            CsvParser, CsvValor
│   ├── Repositories/                   CsvRepository, UsuarioRepository, Repositorios, MediCareDados
│   └── Services/                       ContaService, AgendamentoService, DisponibilidadeService,
│                                       AtendimentoService, NotificacaoService
└── src/MediCare.CsvApp/
    ├── Program.cs                      login e menu inicial
    ├── Sistema.cs, Menu.cs, Entrada.cs, Exibicao.cs
    └── Telas/                          TelaCadastro, FluxoAgendamento, TelaTeleconsulta,
                                        MenuPaciente, MenuRecepcionista, MenuMedico,
                                        MenuAdministrador
```

9.3 Principais métodos desenvolvidos

Autenticação (UC02). Contas de provedor externo são identificadas antes da verificação de senha:

```
public Usuario Autenticar(string email, string senha)
{
    var usuario = ObterPorEmail(email)
        ?? throw new InvalidOperationException("E-mail ou senha inválidos.");

    if (usuario.Provedor != ProvedorAutenticacao.Local)
        throw new InvalidOperationException($"Esta conta utiliza login pelo {usuario.Provedor}.");
    if (!usuario.VerificarSenha(senha))
        throw new InvalidOperationException("E-mail ou senha inválidos.");
    if (!usuario.Ativo)
        throw new InvalidOperationException("Conta desativada. Procure o administrador.");

    return usuario;
}
```

Notificações (UC05 a UC07 e UC14). O destino é escolhido conforme o canal preferido do paciente:

```
public Notificacao Registrar(Consulta consulta, TipoNotificacao tipo)
{
    var paciente = dados.Pacientes.ObterPorId(consulta.PacienteId)!;
    var canal = paciente.CanalPreferido;
    var destino = canal == CanalNotificacao.Email ? paciente.Email : paciente.Telefone;

    var notificacao = new Notificacao(consulta.Id, tipo, canal, destino, MontarMensagem(consulta, tipo));
    dados.Notificacoes.Adicionar(notificacao);
    dados.Notificacoes.Salvar();
    return notificacao;
}
```

Lembretes (UC14). Cada consulta recebe no máximo um lembrete:

```
public int GerarLembretes(DateOnly dia)
{
    var jaLembradas = dados.Notificacoes.ListarTodos()
        .Where(n => n.Tipo == TipoNotificacao.Lembrete)
        .Select(n => n.ConsultaId)
        .ToHashSet();

    var consultas = dados.Consultas.ListarTodos()
        .Where(c => c.Status == StatusConsulta.Agendada
                    && !jaLembradas.Contains(c.Id)
                    && dados.Horarios.ObterPorId(c.HorarioId)!.Data == dia)
        .ToList();

    foreach (var consulta in consultas)
        Registrar(consulta, TipoNotificacao.Lembrete);

    return consultas.Count;
}
```

Conflito de horários (UC09). Dois horários conflitam quando são do mesmo médico, no mesmo dia, e os intervalos se sobrepõem:

```
public bool ConflitaCom(HorarioDisponivel outro) =>
    MedicoId == outro.MedicoId &&
    Data == outro.Data &&
    HoraInicio < outro.HoraFim &&
    outro.HoraInicio < HoraFim;
```

Acesso à teleconsulta (UC11). O acesso só é liberado aos participantes e no horário agendado:

```
public Consulta AcessarTeleconsulta(Usuario usuario, int consultaId, DateTime agora)
{
    var consulta = dados.Consultas.ObterPorId(consultaId)
        ?? throw new InvalidOperationException("Consulta não encontrada.");
    var horario = Horario(consulta);

    var participa = usuario switch
    {
        Paciente p => p.Id == consulta.PacienteId,
        Medico m => m.Id == horario.MedicoId,
        _ => false
    };
    if (!participa)
        throw new InvalidOperationException("Você não participa desta consulta.");
    if (horario.Modalidade != Modalidade.Teleconsulta)
        throw new InvalidOperationException("Esta consulta é presencial.");
    if (consulta.Status != StatusConsulta.Agendada)
        throw new InvalidOperationException("A consulta não está agendada.");

    var inicio = horario.Data.ToDateTime(horario.HoraInicio);
    var fim = horario.Data.ToDateTime(horario.HoraFim);
    if (agora < inicio || agora > fim)
        throw new InvalidOperationException(
            $"A teleconsulta só pode ser acessada no horário agendado ({inicio:dd/MM/yyyy HH:mm} a {fim:HH:mm}).");

    return consulta;
}
```

Histórico do paciente (UC15):

```
public IEnumerable<Consulta> HistoricoDoPaciente(int medicoId, int pacienteId)
{
    var consultasDoPaciente = dados.Consultas.ListarTodos()
        .Where(c => c.PacienteId == pacienteId)
        .ToList();

    if (!consultasDoPaciente.Any(c => Horario(c).MedicoId == medicoId))
        throw new InvalidOperationException(
            "Só é possível ver o histórico de pacientes atendidos por você.");

    return consultasDoPaciente
        .Where(c => c.Status == StatusConsulta.Realizada)
        .OrderByDescending(c => Horario(c).Data).ThenByDescending(c => Horario(c).HoraInicio);
}
```

Colunas comuns de usuários. Cada repositório de usuário informa só as suas colunas próprias:

```
public class PacienteRepository(string caminho) : UsuarioRepository<Paciente>(caminho)
{
    protected override string[] ColunasProprias => ["Telefone", "DataNascimento", "CanalPreferido"];

    protected override void LerCamposProprios(Paciente p, IReadOnlyList<string> c)
    {
        p.Telefone = c[0];
        p.DataNascimento = CsvValor.ParaData(c[1]);
        p.CanalPreferido = CsvValor.ParaEnum<CanalNotificacao>(c[2]);
    }

    protected override IEnumerable<string> CamposProprios(Paciente p) =>
        [p.Telefone, CsvValor.De(p.DataNascimento), CsvValor.De(p.CanalPreferido)];
}
```

9.4 Arquivos CSV

Arquivo	Colunas	Situação
pacientes.csv	Id, Nome, Email, SenhaHash, Provedor, Ativo, DataCadastro, Telefone, DataNascimento, CanalPreferido	Alterado
medicos.csv	Id, Nome, Email, SenhaHash, Provedor, Ativo, DataCadastro, Crm, Telefone, EspecialidadeId	Alterado
recepcionistas.csv	Id, Nome, Email, SenhaHash, Provedor, Ativo, DataCadastro	Novo
administradores.csv	Id, Nome, Email, SenhaHash, Provedor, Ativo, DataCadastro	Alterado
notificacoes.csv	Id, ConsultaId, Tipo, Canal, Destino, Mensagem, DataCriacao, Status, DataEnvio	Novo
especialidades.csv, locais_atendimento.csv, horarios.csv, consultas.csv	Mesmas colunas da TP3	Sem alteração

9.5 Execução

O programa é executado com `dotnet run --project src/MediCare.CsvApp -- dados`. Trecho de uma execução em que a recepcionista agenda uma consulta para uma paciente e, depois, o administrador gera os lembretes e envia as notificações pendentes (linhas longas foram abreviadas com “(...)”):

```
Opção: 1
E-mail: recepcao@medicare.com
Senha: senha123
Bem-vindo(a), Fernanda Alves.

== Recepção ==
...
Opção: 3
E-mail do paciente: ana.souza@email.com
Paciente: Ana Paula Souza
  [1] Carla Mendes — Cardiologia — CRM-RJ 123456
  [2] Daniel Rocha — Dermatologia — CRM-RJ 654321
  [3] Elisa Martins — Pediatria — CRM-SP 112233
Médico: 2
  [4] 03/12/2026 10:00-10:30 — Presencial — Consultório Pele & Saúde, Av. Rio Branco, 45, 8º andar, Rio de Janeiro
  [5] 03/12/2026 16:00-16:30 — Teleconsulta
Horário: 4
Consulta 4 agendada. Confirmação enviada por Email.

Opção: 1
E-mail: admin@medicare.com
Senha: senha123
Bem-vindo(a), Administrador MediCare.

== Administrador ==
...
Opção: 5
1 lembrete(s) gerado(s) para 10/11/2026.
Opção: 6
  ...
  [Email → ana.souza@email.com] Consulta confirmada para 03/12/2026 às 10:00 com Dr(a). Daniel Rocha. Local: (...)
  [Email → ana.souza@email.com] A consulta de 01/12/2026 às 09:00 com Dr(a). Carla Mendes foi cancelada.
  [WhatsApp → (21) 91111-2222] Consulta confirmada para 10/11/2026 às 15:00 com Dr(a). Carla Mendes. (...)
  [WhatsApp → (21) 91111-2222] Lembrete: consulta em 10/11/2026 às 15:00 com Dr(a). Carla Mendes. (...)
```

9.6 Testes realizados

- Fluxos completos de cada perfil: paciente, recepcionista, médico e administrador.
- Bloqueio de login por senha em conta Google e de conta desativada.
- Cancelamento de consulta que não pertence ao paciente, horário já reservado e conflito de horários no cadastro de disponibilidade.
- Remoção de horário já utilizado em consulta, que é recusada.
- Histórico exibido apenas para paciente atendido pelo médico.
- Acesso à teleconsulta liberado no horário agendado e recusado fora dele, em consulta presencial e para quem não participa da consulta.
- Atualização de perfil do paciente e do médico, listagem filtrada e cancelamento de agendamento pelo administrador.
- Geração de lembrete único por consulta e envio das notificações pendentes, com o canal e o destino do paciente.
- Leitura e regravação dos nove arquivos CSV sem nenhuma alteração no conteúdo.

---

10. Conclusão

As mudanças pedidas pelo cliente foram incorporadas em todas as partes do projeto. A recepcionista e os sistemas externos de notificação e de autenticação passaram a fazer parte dos atores; os casos de uso afetados foram reescritos e dois novos foram incluídos. O diagrama de classes e o dicionário de dados receberam as classes Recepcionista e Notificacao, os novos atributos e as novas enumerações.

Os diagramas de sequência e de comunicação definiram as mensagens trocadas em cada caso de uso, e a partir deles foram listados e implementados os métodos das classes de entidade e de serviço. O código foi revisado para refletir o novo modelo, e a separação entre serviços e telas permite que a mesma lógica seja usada pela interface Razor Pages na próxima etapa.

Anexo — Código-fonte

Código completo da solução, organizado por pasta. Os arquivos de projeto (.sln e .csproj) usam a configuração padrão do .NET 8, e os arquivos CSV de exemplo seguem as colunas descritas na seção 9.4.

MediCare.Core/Models/Administrador.cs

```
namespace MediCare.Core.Models;

public class Administrador : Usuario
{
}
```

MediCare.Core/Models/Consulta.cs

```
namespace MediCare.Core.Models;

public class Consulta : IEntidade
{
    public int Id { get; set; }
    public int PacienteId { get; set; }
    public int HorarioId { get; private set; }
    public StatusConsulta Status { get; private set; } = StatusConsulta.Agendada;
    public DateTime DataAgendamento { get; private set; }

    public Consulta(int pacienteId, int horarioId)
    {
        PacienteId = pacienteId;
        HorarioId = horarioId;
        DataAgendamento = DateTime.Now;
    }

    internal Consulta(int id, int pacienteId, int horarioId, StatusConsulta status, DateTime dataAgendamento)
    {
        Id = id;
        PacienteId = pacienteId;
        HorarioId = horarioId;
        Status = status;
        DataAgendamento = dataAgendamento;
    }

    public void Cancelar()
    {
        GarantirAgendada();
        Status = StatusConsulta.Cancelada;
    }

    public void Reagendar(int novoHorarioId)
    {
        GarantirAgendada();
        HorarioId = novoHorarioId;
    }

    public void Concluir()
    {
        GarantirAgendada();
        Status = StatusConsulta.Realizada;
    }

    private void GarantirAgendada()
    {
        if (Status != StatusConsulta.Agendada)
            throw new InvalidOperationException(
                $"A consulta está {Status.ToString().ToLower()} e não pode ser alterada.");
    }
}
```

MediCare.Core/Models/Enums.cs

```
namespace MediCare.Core.Models;

public enum Modalidade
{
    Presencial,
    Teleconsulta
}
public enum StatusConsulta
```

```
{
    Agendada,
    Cancelada,
    Realizada
}

public enum ProvedorAutenticacao
{
    Local,
    Google,
    Microsoft,
    Meta
}

public enum CanalNotificacao
{
    Email,
    Sms,
    WhatsApp
}

public enum TipoNotificacao
{
    Confirmacao,
    Reagendamento,
    Cancelamento,
    Lembrete
}

public enum StatusNotificacao
{
    Pendente,
    Enviada
}
```

MediCare.Core/Models/Especialidade.cs

```
namespace MediCare.Core.Models;

public class Especialidade : IEntidade
{
    public int Id { get; set; }
    public string Nome { get; set; } = "";
}
```

MediCare.Core/Models/HorarioDisponivel.cs

```
namespace MediCare.Core.Models;

public class HorarioDisponivel : IEntidade
{
    public int Id { get; set; }
    public int MedicoId { get; set; }
    public DateOnly Data { get; set; }
    public TimeOnly HoraInicio { get; set; }
    public TimeOnly HoraFim { get; set; }
    public Modalidade Modalidade { get; set; }

    // Preenchido somente quando a modalidade é presencial.
    public int? LocalId { get; set; }

    public bool Disponivel { get; private set; } = true;

    public void Reservar()
    {
        if (!Disponivel)
            throw new InvalidOperationException("Este horário já está reservado.");
        Disponivel = false;
    }

    public void Liberar() => Disponivel = true;

    public bool ConflitaCom(HorarioDisponivel outro) =>
        MedicoId == outro.MedicoId &&
        Data == outro.Data &&
        HoraInicio < outro.HoraFim &&
        outro.HoraInicio < HoraFim;

    internal void Restaurar(bool disponivel) => Disponivel = disponivel;
}
```

MediCare.Core/Models/IEntidade.cs

```
namespace MediCare.Core.Models;
```

```
public interface IEntidade
{
    int Id { get; set; }
}
```

MediCare.Core/Models/LocalAtendimento.cs

```
namespace MediCare.Core.Models;

public class LocalAtendimento : IEntidade
{
    public int Id { get; set; }
    public int MedicoId { get; set; }
    public string Nome { get; set; } = "";
    public string Endereco { get; set; } = "";
    public string Cidade { get; set; } = "";
}
```

MediCare.Core/Models/Medico.cs

```
namespace MediCare.Core.Models;

public class Medico : Usuario
{
    public string Crm { get; set; } = "";
    public string Telefone { get; set; } = "";
    public int EspecialidadeId { get; set; }
}
```

MediCare.Core/Models/Notificacao.cs

```
namespace MediCare.Core.Models;

/// <summary>
/// Mensagem gerada pelo sistema para ser entregue ao paciente pelo sistema de notificação
/// (e-mail, SMS ou WhatsApp).
/// </summary>
public class Notificacao : IEntidade
{
    public int Id { get; set; }
    public int ConsultaId { get; private set; }
    public TipoNotificacao Tipo { get; private set; }
    public CanalNotificacao Canal { get; private set; }
    public string Destino { get; private set; }
    public string Mensagem { get; private set; }
    public DateTime DataCriacao { get; private set; }
    public StatusNotificacao Status { get; private set; } = StatusNotificacao.Pendente;
    public DateTime? DataEnvio { get; private set; }

    public Notificacao(int consultaId, TipoNotificacao tipo, CanalNotificacao canal, string destino, string mensagem)
    {
        ConsultaId = consultaId;
        Tipo = tipo;
        Canal = canal;
        Destino = destino;
        Mensagem = mensagem;
        DataCriacao = DateTime.Now;
    }

    internal Notificacao(int id, int consultaId, TipoNotificacao tipo, CanalNotificacao canal, string destino,
        string mensagem, DateTime dataCriacao, StatusNotificacao status, DateTime? dataEnvio)
        : this(consultaId, tipo, canal, destino, mensagem)
    {
        Id = id;
        DataCriacao = dataCriacao;
        Status = status;
        DataEnvio = dataEnvio;
    }

    public void MarcarComoEnviada()
    {
        if (Status == StatusNotificacao.Enviada)
            throw new InvalidOperationException("A notificação já foi enviada.");
        Status = StatusNotificacao.Enviada;
        DataEnvio = DateTime.Now;
    }
}
```

MediCare.Core/Models/Paciente.cs

```
namespace MediCare.Core.Models;

public class Paciente : Usuario
```

```
{
    public string Telefone { get; set; } = "";
    public DateOnly DataNascimento { get; set; }
    public CanalNotificacao CanalPreferido { get; set; } = CanalNotificacao.Email;
}
```

MediCare.Core/Models/Recepcionista.cs

```
namespace MediCare.Core.Models;

public class Recepcionista : Usuario
{
}
```

MediCare.Core/Models/Usuario.cs

```
using System.Security.Cryptography;
using System.Text;

namespace MediCare.Core.Models;

public abstract class Usuario : IEntidade
{
    public int Id { get; set; }
    public string Nome { get; set; } = "";
    public string Email { get; set; } = "";
    public string SenhaHash { get; private set; } = "";
    public ProvedorAutenticacao Provedor { get; set; } = ProvedorAutenticacao.Local;
    public bool Ativo { get; private set; } = true;
    public DateTime DataCadastro { get; set; } = DateTime.Now;

    public void DefinirSenha(string senha)
    {
        if (string.IsNullOrWhiteSpace(senha))
            throw new ArgumentException("A senha não pode ser vazia.");
        SenhaHash = GerarHash(senha);
    }

    public bool VerificarSenha(string senha) =>
        Provedor == ProvedorAutenticacao.Local && SenhaHash == GerarHash(senha);

    public void Desativar() => Ativo = false;
    public void Ativar() => Ativo = true;

    // Usado apenas ao reconstruir o objeto a partir do CSV.
    internal void Restaurar(string senhaHash, bool ativo)
    {
        SenhaHash = senhaHash;
        Ativo = ativo;
    }

    private static string GerarHash(string senha) =>
        Convert.ToHexString(SHA256.HashData(Encoding.UTF8.GetBytes(senha)));
}
```

MediCare.Core/Csv/CsvParser.cs

```
using System.Text;

namespace MediCare.Core.Csv;

/// <summary>
/// Leitura e escrita de CSV no formato RFC 4180: campos separados por vírgula,
/// e campos com vírgula, aspas ou quebra de linha ficam entre aspas duplas.
/// </summary>
public static class CsvParser
{
    public const char Separador = ',';

    public static List<List<string>> Ler(TextReader leitor)
    {
        var registros = new List<List<string>>();
        var registro = new List<string>();
        var campo = new StringBuilder();
        var entreAspas = false;
        int c;

        while ((c = leitor.Read()) != -1)
        {
            var ch = (char)c;

            if (entreAspas)
            {
                if (ch == '"')
```

```
                {
                    if (leitor.Peek() == '"')
                    {
                        campo.Append('"');
                        leitor.Read();
                    }
                    else
                    {
                        entreAspas = false;
                    }
                }
                else
                {
                    campo.Append(ch);
                }
                continue;
            }

            switch (ch)
            {
                case '"':
                    entreAspas = true;
                    break;
                case Separador:
                    registro.Add(campo.ToString());
                    campo.Clear();
                    break;
                case '\r':
                    break;
                case '\n':
                    registro.Add(campo.ToString());
                    campo.Clear();
                    AdicionarSeNaoVazio(registros, registro);
                    registro = new List<string>();
                    break;
                default:
                    campo.Append(ch);
                    break;
            }
        }

        if (entreAspas)
            throw new FormatException("Arquivo CSV inválido: aspas não fechadas.");

        if (campo.Length > 0 || registro.Count > 0)
        {
            registro.Add(campo.ToString());
            AdicionarSeNaoVazio(registros, registro);
        }

        return registros;
    }

    public static string FormatarLinha(IEnumerable<string> campos) =>
        string.Join(Separador, campos.Select(Escapar));

    private static string Escapar(string valor)
    {
        if (valor.IndexOfAny(new[] { Separador, '"', '\n', '\r' }) < 0)
            return valor;
        return $"\"{valor.Replace("\"", "\"\"")}\"";
    }

    private static void AdicionarSeNaoVazio(List<List<string>> registros, List<string> registro)
    {
        var linhaEmBranco = registro.Count == 1 && registro[0].Length == 0;
        if (!linhaEmBranco)
            registros.Add(registro);
    }
}
```

MediCare.Core/Csv/CsvValor.cs

```
using System.Globalization;

namespace MediCare.Core.Csv;

/// <summary>
/// Conversões entre texto do CSV e tipos .NET, sempre em formato invariável
/// (datas ISO, ponto como separador decimal), para o arquivo não depender
/// da configuração regional da máquina.
/// </summary>
public static class CsvValor
{
    private const string FormatoData = "yyyy-MM-dd";
```

```
    private const string FormatoHora = "HH:mm";
    private const string FormatoDataHora = "yyyy-MM-ddTHH:mm:ss";
    private static readonly CultureInfo Cultura = CultureInfo.InvariantCulture;

    public static string De(int valor) => valor.ToString(Cultura);
    public static string De(int? valor) => valor?.ToString(Cultura) ?? "";
    public static string De(bool valor) => valor ? "true" : "false";
    public static string De(DateOnly valor) => valor.ToString(FormatoData, Cultura);
    public static string De(TimeOnly valor) => valor.ToString(FormatoHora, Cultura);
    public static string De(DateTime valor) => valor.ToString(FormatoDataHora, Cultura);
    public static string De(DateTime? valor) => valor is null ? "" : De(valor.Value);
    public static string De<TEnum>(TEnum valor) where TEnum : struct, Enum => valor.ToString();

    public static int ParaInt(string texto) => int.Parse(texto, Cultura);
    public static int? ParaIntOpcional(string texto) =>
        string.IsNullOrWhiteSpace(texto) ? null : int.Parse(texto, Cultura);
    public static bool ParaBool(string texto) => bool.Parse(texto);
    public static DateOnly ParaData(string texto) => DateOnly.ParseExact(texto, FormatoData, Cultura);
    public static TimeOnly ParaHora(string texto) => TimeOnly.ParseExact(texto, FormatoHora, Cultura);
    public static DateTime ParaDataHora(string texto) => DateTime.ParseExact(texto, FormatoDataHora, Cultura);
    public static DateTime? ParaDataHoraOpcional(string texto) =>
        string.IsNullOrWhiteSpace(texto) ? null : ParaDataHora(texto);
    public static TEnum ParaEnum<TEnum>(string texto) where TEnum : struct, Enum =>
        Enum.Parse<TEnum>(texto, ignoreCase: true);
}
```

MediCare.Core/Repositories/CsvRepository.cs

```
using System.Text;
using MediCare.Core.Csv;
using MediCare.Core.Models;

namespace MediCare.Core.Repositories;

/// <summary>
/// Repositório genérico que mantém uma entidade em um arquivo CSV.
/// Os registros são carregados na criação do repositório e gravados em Salvar().
/// Cada classe filha define o cabeçalho e como converter uma linha em objeto e vice-versa.
/// </summary>
public abstract class CsvRepository<T> where T : class, IEntidade
{
    private readonly string _caminho;
    private readonly List<T> _itens = new();

    protected CsvRepository(string caminho)
    {
        _caminho = caminho;
        Carregar();
    }

    protected abstract string[] Cabecalho { get; }
    protected abstract T DeCampos(IReadOnlyList<string> campos);
    protected abstract IEnumerable<string> ParaCampos(T item);

    public IReadOnlyList<T> ListarTodos() => _itens;

    public T? ObterPorId(int id) => _itens.FirstOrDefault(i => i.Id == id);

    public T Adicionar(T item)
    {
        item.Id = _itens.Count == 0 ? 1 : _itens.Max(i => i.Id) + 1;
        _itens.Add(item);
        return item;
    }

    public bool Remover(int id) => _itens.RemoveAll(i => i.Id == id) > 0;

    public void Salvar()
    {
        Directory.CreateDirectory(Path.GetDirectoryName(Path.GetFullPath(_caminho))!);

        var conteudo = new StringBuilder();
        conteudo.AppendLine(CsvParser.FormatarLinha(Cabecalho));
        foreach (var item in _itens.OrderBy(i => i.Id))
            conteudo.AppendLine(CsvParser.FormatarLinha(ParaCampos(item)));

        // Grava primeiro em um arquivo temporário para não corromper o original em caso de falha.
        var temporario = _caminho + ".tmp";
        File.WriteAllText(temporario, conteudo.ToString(), new UTF8Encoding(false));
        File.Move(temporario, _caminho, overwrite: true);
    }

    private void Carregar()
    {
        if (!File.Exists(_caminho))
```

```
            return;

        using var leitor = new StreamReader(_caminho, Encoding.UTF8);
        var registros = CsvParser.Ler(leitor);
        if (registros.Count == 0)
            return;

        ValidarCabecalho(registros[0]);

        for (var i = 1; i < registros.Count; i++)
        {
            var campos = registros[i];
            if (campos.Count != Cabecalho.Length)
                throw Erro(i, $"esperados {Cabecalho.Length} campos, encontrados {campos.Count}");

            try
            {
                _itens.Add(DeCampos(campos));
            }
            catch (FormatException ex)
            {
                throw Erro(i, ex.Message);
            }
        }
    }

    private void ValidarCabecalho(IReadOnlyList<string> cabecalhoArquivo)
    {
        if (!cabecalhoArquivo.SequenceEqual(Cabecalho, StringComparer.OrdinalIgnoreCase))
            throw new InvalidDataException(
                $"Cabeçalho inválido em {Path.GetFileName(_caminho)}. Esperado: {string.Join(",", Cabecalho)}");
    }

    private InvalidDataException Erro(int registro, string detalhe) =>
        new($"{Path.GetFileName(_caminho)}, registro {registro}: {detalhe}");
}
```

MediCare.Core/Repositories/MediCareDados.cs

```
using MediCare.Core.Models;

namespace MediCare.Core.Repositories;

/// <summary>
/// Reúne os repositórios de todas as entidades em uma pasta de dados.
/// </summary>
public class MediCareDados(string pasta)
{
    public EspecialidadeRepository Especialidades { get; } = new(Path.Combine(pasta, "especialidades.csv"));
    public PacienteRepository Pacientes { get; } = new(Path.Combine(pasta, "pacientes.csv"));
    public MedicoRepository Medicos { get; } = new(Path.Combine(pasta, "medicos.csv"));
    public RecepcionistaRepository Recepcionistas { get; } = new(Path.Combine(pasta, "recepcionistas.csv"));
    public AdministradorRepository Administradores { get; } = new(Path.Combine(pasta, "administradores.csv"));
    public LocalAtendimentoRepository Locais { get; } = new(Path.Combine(pasta, "locais_atendimento.csv"));
    public HorarioRepository Horarios { get; } = new(Path.Combine(pasta, "horarios.csv"));
    public ConsultaRepository Consultas { get; } = new(Path.Combine(pasta, "consultas.csv"));
    public NotificacaoRepository Notificacoes { get; } = new(Path.Combine(pasta, "notificacoes.csv"));

    public IEnumerable<Usuario> TodosOsUsuarios() =>
        Pacientes.ListarTodos().Cast<Usuario>()
            .Concat(Medicos.ListarTodos())
            .Concat(Recepcionistas.ListarTodos())
            .Concat(Administradores.ListarTodos());

    public void SalvarUsuarios()
    {
        Pacientes.Salvar();
        Medicos.Salvar();
        Recepcionistas.Salvar();
        Administradores.Salvar();
    }
}
```

MediCare.Core/Repositories/Repositorios.cs

```
using MediCare.Core.Csv;
using MediCare.Core.Models;

namespace MediCare.Core.Repositories;

public class PacienteRepository(string caminho) : UsuarioRepository<Paciente>(caminho)
{
    protected override string[] ColunasProprias => ["Telefone", "DataNascimento", "CanalPreferido"];
    protected override void LerCamposProprios(Paciente p, IReadOnlyList<string> c)
```

```
    {
        p.Telefone = c[0];
        p.DataNascimento = CsvValor.ParaData(c[1]);
        p.CanalPreferido = CsvValor.ParaEnum<CanalNotificacao>(c[2]);
    }

    protected override IEnumerable<string> CamposProprios(Paciente p) =>
        [p.Telefone, CsvValor.De(p.DataNascimento), CsvValor.De(p.CanalPreferido)];
}

public class MedicoRepository(string caminho) : UsuarioRepository<Medico>(caminho)
{
    protected override string[] ColunasProprias => ["Crm", "Telefone", "EspecialidadeId"];

    protected override void LerCamposProprios(Medico m, IReadOnlyList<string> c)
    {
        m.Crm = c[0];
        m.Telefone = c[1];
        m.EspecialidadeId = CsvValor.ParaInt(c[2]);
    }

    protected override IEnumerable<string> CamposProprios(Medico m) =>
        [m.Crm, m.Telefone, CsvValor.De(m.EspecialidadeId)];
}

public class RecepcionistaRepository(string caminho) : UsuarioRepository<Recepcionista>(caminho);

public class AdministradorRepository(string caminho) : UsuarioRepository<Administrador>(caminho);

public class EspecialidadeRepository(string caminho) : CsvRepository<Especialidade>(caminho)
{
    protected override string[] Cabecalho => ["Id", "Nome"];

    protected override Especialidade DeCampos(IReadOnlyList<string> c) => new()
    {
        Id = CsvValor.ParaInt(c[0]),
        Nome = c[1]
    };

    protected override IEnumerable<string> ParaCampos(Especialidade e) => [CsvValor.De(e.Id), e.Nome];
}

public class LocalAtendimentoRepository(string caminho) : CsvRepository<LocalAtendimento>(caminho)
{
    protected override string[] Cabecalho => ["Id", "MedicoId", "Nome", "Endereco", "Cidade"];

    protected override LocalAtendimento DeCampos(IReadOnlyList<string> c) => new()
    {
        Id = CsvValor.ParaInt(c[0]),
        MedicoId = CsvValor.ParaInt(c[1]),
        Nome = c[2],
        Endereco = c[3],
        Cidade = c[4]
    };

    protected override IEnumerable<string> ParaCampos(LocalAtendimento l) =>
        [CsvValor.De(l.Id), CsvValor.De(l.MedicoId), l.Nome, l.Endereco, l.Cidade];
}

public class HorarioRepository(string caminho) : CsvRepository<HorarioDisponivel>(caminho)
{
    protected override string[] Cabecalho =>
        ["Id", "MedicoId", "Data", "HoraInicio", "HoraFim", "Modalidade", "LocalId", "Disponivel"];

    protected override HorarioDisponivel DeCampos(IReadOnlyList<string> c)
    {
        var horario = new HorarioDisponivel
        {
            Id = CsvValor.ParaInt(c[0]),
            MedicoId = CsvValor.ParaInt(c[1]),
            Data = CsvValor.ParaData(c[2]),
            HoraInicio = CsvValor.ParaHora(c[3]),
            HoraFim = CsvValor.ParaHora(c[4]),
            Modalidade = CsvValor.ParaEnum<Modalidade>(c[5]),
            LocalId = CsvValor.ParaIntOpcional(c[6])
        };
        horario.Restaurar(CsvValor.ParaBool(c[7]));
        return horario;
    }

    protected override IEnumerable<string> ParaCampos(HorarioDisponivel h) =>
    [
        CsvValor.De(h.Id), CsvValor.De(h.MedicoId), CsvValor.De(h.Data), CsvValor.De(h.HoraInicio),
        CsvValor.De(h.HoraFim), CsvValor.De(h.Modalidade), CsvValor.De(h.LocalId), CsvValor.De(h.Disponivel)
    ];
}
```

```
public class ConsultaRepository(string caminho) : CsvRepository<Consulta>(caminho)
{
    protected override string[] Cabecalho => ["Id", "PacienteId", "HorarioId", "Status", "DataAgendamento"];

    protected override Consulta DeCampos(IReadOnlyList<string> c) => new(
        id: CsvValor.ParaInt(c[0]),
        pacienteId: CsvValor.ParaInt(c[1]),
        horarioId: CsvValor.ParaInt(c[2]),
        status: CsvValor.ParaEnum<StatusConsulta>(c[3]),
        dataAgendamento: CsvValor.ParaDataHora(c[4]));

    protected override IEnumerable<string> ParaCampos(Consulta c) =>
    [
        CsvValor.De(c.Id), CsvValor.De(c.PacienteId), CsvValor.De(c.HorarioId),
        CsvValor.De(c.Status), CsvValor.De(c.DataAgendamento)
    ];
}

public class NotificacaoRepository(string caminho) : CsvRepository<Notificacao>(caminho)
{
    protected override string[] Cabecalho =>
        ["Id", "ConsultaId", "Tipo", "Canal", "Destino", "Mensagem", "DataCriacao", "Status", "DataEnvio"];

    protected override Notificacao DeCampos(IReadOnlyList<string> c) => new(
        id: CsvValor.ParaInt(c[0]),
        consultaId: CsvValor.ParaInt(c[1]),
        tipo: CsvValor.ParaEnum<TipoNotificacao>(c[2]),
        canal: CsvValor.ParaEnum<CanalNotificacao>(c[3]),
        destino: c[4],
        mensagem: c[5],
        dataCriacao: CsvValor.ParaDataHora(c[6]),
        status: CsvValor.ParaEnum<StatusNotificacao>(c[7]),
        dataEnvio: CsvValor.ParaDataHoraOpcional(c[8]));

    protected override IEnumerable<string> ParaCampos(Notificacao n) =>
    [
        CsvValor.De(n.Id), CsvValor.De(n.ConsultaId), CsvValor.De(n.Tipo), CsvValor.De(n.Canal), n.Destino,
        n.Mensagem, CsvValor.De(n.DataCriacao), CsvValor.De(n.Status), CsvValor.De(n.DataEnvio)
    ];
}
```

MediCare.Core/Repositories/UsuarioRepository.cs

```
using MediCare.Core.Csv;
using MediCare.Core.Models;

namespace MediCare.Core.Repositories;

/// <summary>
/// Base dos repositórios de usuários. Trata as colunas herdadas de Usuario;
/// as classes filhas acrescentam apenas as colunas próprias de cada tipo.
/// </summary>
public abstract class UsuarioRepository<T>(string caminho) : CsvRepository<T>(caminho)
    where T : Usuario, new()
{
    private static readonly string[] ColunasUsuario =
        ["Id", "Nome", "Email", "SenhaHash", "Provedor", "Ativo", "DataCadastro"];

    protected virtual string[] ColunasProprias => [];

    protected override string[] Cabecalho => [.. ColunasUsuario, .. ColunasProprias];

    protected virtual void LerCamposProprios(T usuario, IReadOnlyList<string> campos) { }

    protected virtual IEnumerable<string> CamposProprios(T usuario) => [];

    protected override T DeCampos(IReadOnlyList<string> c)
    {
        var usuario = new T
        {
            Id = CsvValor.ParaInt(c[0]),
            Nome = c[1],
            Email = c[2],
            Provedor = CsvValor.ParaEnum<ProvedorAutenticacao>(c[4]),
            DataCadastro = CsvValor.ParaDataHora(c[6])
        };
        usuario.Restaurar(c[3], CsvValor.ParaBool(c[5]));
        LerCamposProprios(usuario, c.Skip(ColunasUsuario.Length).ToList());
        return usuario;
    }

    protected override IEnumerable<string> ParaCampos(T u) =>
    [
        CsvValor.De(u.Id), u.Nome, u.Email, u.SenhaHash, CsvValor.De(u.Provedor),
        CsvValor.De(u.Ativo), CsvValor.De(u.DataCadastro), .. CamposProprios(u)
```

```
    ];
}
```

MediCare.Core/Services/AgendamentoService.cs

```
using MediCare.Core.Models;
using MediCare.Core.Repositories;

namespace MediCare.Core.Services;

/// <summary>
/// Casos de uso UC03 a UC07, usados pelo paciente e pela recepcionista
/// (pesquisar médicos, consultar horários, agendar, cancelar e reagendar consultas),
/// e UC13, usado pelo administrador para acompanhar e cancelar agendamentos.
/// </summary>
public class AgendamentoService(MediCareDados dados, NotificacaoService notificacoes)
{
    public IEnumerable<Medico> PesquisarMedicos(int? especialidadeId = null) =>
        dados.Medicos.ListarTodos()
            .Where(m => m.Ativo && (especialidadeId is null || m.EspecialidadeId == especialidadeId));

    public IEnumerable<HorarioDisponivel> HorariosDisponiveis(int medicoId) =>
        dados.Horarios.ListarTodos()
            .Where(h => h.MedicoId == medicoId && h.Disponivel)
            .OrderBy(h => h.Data).ThenBy(h => h.HoraInicio);

    public IEnumerable<Consulta> ConsultasDoPaciente(int pacienteId) =>
        dados.Consultas.ListarTodos().Where(c => c.PacienteId == pacienteId);

    public IEnumerable<Consulta> ListarAgendamentos(StatusConsulta? status = null) =>
        dados.Consultas.ListarTodos()
            .Where(c => status is null || c.Status == status)
            .OrderBy(c => ObterHorario(c.HorarioId).Data)
            .ThenBy(c => ObterHorario(c.HorarioId).HoraInicio);

    public Consulta Agendar(int pacienteId, int horarioId)
    {
        var paciente = dados.Pacientes.ObterPorId(pacienteId)
            ?? throw new InvalidOperationException("Paciente não encontrado.");
        if (!paciente.Ativo)
            throw new InvalidOperationException("Paciente desativado não pode agendar consultas.");

        var horario = ObterHorario(horarioId);
        horario.Reservar();

        var consulta = dados.Consultas.Adicionar(new Consulta(pacienteId, horarioId));
        Salvar();
        notificacoes.Registrar(consulta, TipoNotificacao.Confirmacao);
        return consulta;
    }

    public void Cancelar(int consultaId)
    {
        var consulta = ObterConsulta(consultaId);
        consulta.Cancelar();
        ObterHorario(consulta.HorarioId).Liberar();
        Salvar();
        notificacoes.Registrar(consulta, TipoNotificacao.Cancelamento);
    }

    public void Reagendar(int consultaId, int novoHorarioId)
    {
        var consulta = ObterConsulta(consultaId);
        var horarioAtual = ObterHorario(consulta.HorarioId);
        var novoHorario = ObterHorario(novoHorarioId);

        if (novoHorario.MedicoId != horarioAtual.MedicoId)
            throw new InvalidOperationException("O novo horário deve ser do mesmo médico.");
        if (!novoHorario.Disponivel)
            throw new InvalidOperationException("O novo horário não está disponível.");

        consulta.Reagendar(novoHorarioId);
        novoHorario.Reservar();
        horarioAtual.Liberar();
        Salvar();
        notificacoes.Registrar(consulta, TipoNotificacao.Reagendamento);
    }

    private Consulta ObterConsulta(int id) =>
        dados.Consultas.ObterPorId(id) ?? throw new InvalidOperationException("Consulta não encontrada.");

    private HorarioDisponivel ObterHorario(int id) =>
        dados.Horarios.ObterPorId(id) ?? throw new InvalidOperationException("Horário não encontrado.");

    private void Salvar()
```

```
    {
        dados.Horarios.Salvar();
        dados.Consultas.Salvar();
    }
}
```

MediCare.Core/Services/AtendimentoService.cs

```
using MediCare.Core.Models;
using MediCare.Core.Repositories;

namespace MediCare.Core.Services;

/// <summary>
/// Casos de uso de atendimento: UC10 (gerenciar agenda), UC11 (realizar teleconsulta)
/// e UC15 (consultar histórico do paciente).
/// </summary>
public class AtendimentoService(MediCareDados dados)
{
    public IEnumerable<Consulta> Agenda(int medicoId) =>
        dados.Consultas.ListarTodos()
            .Where(c => c.Status == StatusConsulta.Agendada && Horario(c).MedicoId == medicoId)
            .OrderBy(c => Horario(c).Data).ThenBy(c => Horario(c).HoraInicio);

    public void ConcluirConsulta(int medicoId, int consultaId)
    {
        var consulta = dados.Consultas.ObterPorId(consultaId);
        if (consulta is null || Horario(consulta).MedicoId != medicoId)
            throw new InvalidOperationException("Consulta não encontrada na sua agenda.");

        consulta.Concluir();
        dados.Consultas.Salvar();
    }

    public IEnumerable<Consulta> HistoricoDoPaciente(int medicoId, int pacienteId)
    {
        var consultasDoPaciente = dados.Consultas.ListarTodos()
            .Where(c => c.PacienteId == pacienteId)
            .ToList();

        if (!consultasDoPaciente.Any(c => Horario(c).MedicoId == medicoId))
            throw new InvalidOperationException(
                "Só é possível ver o histórico de pacientes atendidos por você.");

        return consultasDoPaciente
            .Where(c => c.Status == StatusConsulta.Realizada)
            .OrderByDescending(c => Horario(c).Data).ThenByDescending(c => Horario(c).HoraInicio);
    }

    public Consulta AcessarTeleconsulta(Usuario usuario, int consultaId, DateTime agora)
    {
        var consulta = dados.Consultas.ObterPorId(consultaId)
            ?? throw new InvalidOperationException("Consulta não encontrada.");
        var horario = Horario(consulta);

        var participa = usuario switch
        {
            Paciente p => p.Id == consulta.PacienteId,
            Medico m => m.Id == horario.MedicoId,
            _ => false
        };
        if (!participa)
            throw new InvalidOperationException("Você não participa desta consulta.");
        if (horario.Modalidade != Modalidade.Teleconsulta)
            throw new InvalidOperationException("Esta consulta é presencial.");
        if (consulta.Status != StatusConsulta.Agendada)
            throw new InvalidOperationException("A consulta não está agendada.");

        var inicio = horario.Data.ToDateTime(horario.HoraInicio);
        var fim = horario.Data.ToDateTime(horario.HoraFim);
        if (agora < inicio || agora > fim)
            throw new InvalidOperationException(
                $"A teleconsulta só pode ser acessada no horário agendado ({inicio:dd/MM/yyyy HH:mm} a {fim:HH:mm}).");

        return consulta;
    }

    public IEnumerable<Consulta> TeleconsultasAgendadas(Usuario usuario) =>
        dados.Consultas.ListarTodos()
            .Where(c => c.Status == StatusConsulta.Agendada && Horario(c).Modalidade == Modalidade.Teleconsulta)
            .Where(c => usuario switch
            {
                Paciente p => c.PacienteId == p.Id,
                Medico m => Horario(c).MedicoId == m.Id,
                _ => false
```

```
            });

    private HorarioDisponivel Horario(Consulta consulta) => dados.Horarios.ObterPorId(consulta.HorarioId)!;
}
```

MediCare.Core/Services/ContaService.cs

```
using MediCare.Core.Models;
using MediCare.Core.Repositories;

namespace MediCare.Core.Services;

/// <summary>
/// Contas de usuário: UC01 (criar conta), UC02 (login), UC08 (gerenciar perfil) e UC12 (gerenciar usuários).
/// </summary>
public class ContaService(MediCareDados dados)
{
    public Paciente CadastrarPaciente(string nome, string email, string senha, string telefone,
        DateOnly dataNascimento, CanalNotificacao canal)
    {
        GarantirEmailLivre(email);

        var paciente = new Paciente
        {
            Nome = nome,
            Email = email,
            Telefone = telefone,
            DataNascimento = dataNascimento,
            CanalPreferido = canal
        };
        paciente.DefinirSenha(senha);

        dados.Pacientes.Adicionar(paciente);
        dados.Pacientes.Salvar();
        return paciente;
    }

    public Medico CadastrarMedico(string nome, string email, string senha, string crm, string telefone,
        int especialidadeId)
    {
        GarantirEmailLivre(email);
        if (dados.Especialidades.ObterPorId(especialidadeId) is null)
            throw new ArgumentException("Especialidade não encontrada.");

        var medico = new Medico
        {
            Nome = nome,
            Email = email,
            Crm = crm,
            Telefone = telefone,
            EspecialidadeId = especialidadeId
        };
        medico.DefinirSenha(senha);

        dados.Medicos.Adicionar(medico);
        dados.Medicos.Salvar();
        return medico;
    }

    public Usuario Autenticar(string email, string senha)
    {
        var usuario = ObterPorEmail(email)
            ?? throw new InvalidOperationException("E-mail ou senha inválidos.");

        if (usuario.Provedor != ProvedorAutenticacao.Local)
            throw new InvalidOperationException($"Esta conta utiliza login pelo {usuario.Provedor}.");
        if (!usuario.VerificarSenha(senha))
            throw new InvalidOperationException("E-mail ou senha inválidos.");
        if (!usuario.Ativo)
            throw new InvalidOperationException("Conta desativada. Procure o administrador.");

        return usuario;
    }

    public void AtualizarPerfil(Paciente paciente, string telefone, CanalNotificacao canal)
    {
        paciente.Telefone = telefone;
        paciente.CanalPreferido = canal;
        dados.Pacientes.Salvar();
    }

    public void AtualizarPerfil(Medico medico, string telefone)
    {
        medico.Telefone = telefone;
        dados.Medicos.Salvar();
```

```
    }

    public Usuario? ObterPorEmail(string email) =>
        dados.TodosOsUsuarios().FirstOrDefault(u => u.Email.Equals(email, StringComparison.OrdinalIgnoreCase));

    public IEnumerable<Usuario> ListarUsuarios() => dados.TodosOsUsuarios();

    public void AlterarSituacao(string email, bool ativo)
    {
        var usuario = ObterPorEmail(email) ?? throw new InvalidOperationException("Usuário não encontrado.");
        if (ativo) usuario.Ativar();
        else usuario.Desativar();
        dados.SalvarUsuarios();
    }

    private void GarantirEmailLivre(string email)
    {
        if (ObterPorEmail(email) is not null)
            throw new InvalidOperationException("Já existe uma conta com este e-mail.");
    }
}
```

MediCare.Core/Services/DisponibilidadeService.cs

```
using MediCare.Core.Models;
using MediCare.Core.Repositories;

namespace MediCare.Core.Services;

/// <summary>
/// Caso de uso UC09: o médico cadastra e remove seus horários de atendimento.
/// </summary>
public class DisponibilidadeService(MediCareDados dados)
{
    public IEnumerable<LocalAtendimento> LocaisDoMedico(int medicoId) =>
        dados.Locais.ListarTodos().Where(l => l.MedicoId == medicoId);

    public IEnumerable<HorarioDisponivel> HorariosDoMedico(int medicoId) =>
        dados.Horarios.ListarTodos()
            .Where(h => h.MedicoId == medicoId)
            .OrderBy(h => h.Data).ThenBy(h => h.HoraInicio);

    public HorarioDisponivel CadastrarHorario(int medicoId, DateOnly data, TimeOnly inicio, TimeOnly fim,
        Modalidade modalidade, int? localId)
    {
        if (data < DateOnly.FromDateTime(DateTime.Today))
            throw new ArgumentException("Não é possível cadastrar horários em datas passadas.");
        if (fim <= inicio)
            throw new ArgumentException("A hora de término deve ser posterior à hora de início.");

        if (modalidade == Modalidade.Presencial)
        {
            var local = localId is int id ? dados.Locais.ObterPorId(id) : null;
            if (local is null || local.MedicoId != medicoId)
                throw new ArgumentException("Informe um local de atendimento cadastrado para este médico.");
        }
        else
        {
            localId = null;
        }

        var horario = new HorarioDisponivel
        {
            MedicoId = medicoId,
            Data = data,
            HoraInicio = inicio,
            HoraFim = fim,
            Modalidade = modalidade,
            LocalId = localId
        };

        if (HorariosDoMedico(medicoId).Any(h => h.ConflitaCom(horario)))
            throw new InvalidOperationException("O horário conflita com outro já cadastrado.");

        dados.Horarios.Adicionar(horario);
        dados.Horarios.Salvar();
        return horario;
    }

    public void RemoverHorario(int medicoId, int horarioId)
    {
        var horario = dados.Horarios.ObterPorId(horarioId);
        if (horario is null || horario.MedicoId != medicoId)
            throw new InvalidOperationException("Horário não encontrado.");
        if (dados.Consultas.ListarTodos().Any(c => c.HorarioId == horarioId))
```

```
            throw new InvalidOperationException("O horário possui consultas registradas e não pode ser removido.");

        dados.Horarios.Remover(horarioId);
        dados.Horarios.Salvar();
    }
}
```

MediCare.Core/Services/NotificacaoService.cs

```
using MediCare.Core.Models;
using MediCare.Core.Repositories;

namespace MediCare.Core.Services;

/// <summary>
/// Gera as notificações de consulta (UC14) e controla o que já foi entregue ao sistema de notificação.
/// </summary>
public class NotificacaoService(MediCareDados dados)
{
    public Notificacao Registrar(Consulta consulta, TipoNotificacao tipo)
    {
        var paciente = dados.Pacientes.ObterPorId(consulta.PacienteId)!;
        var canal = paciente.CanalPreferido;
        var destino = canal == CanalNotificacao.Email ? paciente.Email : paciente.Telefone;

        var notificacao = new Notificacao(consulta.Id, tipo, canal, destino, MontarMensagem(consulta, tipo));
        dados.Notificacoes.Adicionar(notificacao);
        dados.Notificacoes.Salvar();
        return notificacao;
    }

    public int GerarLembretes(DateOnly dia)
    {
        var jaLembradas = dados.Notificacoes.ListarTodos()
            .Where(n => n.Tipo == TipoNotificacao.Lembrete)
            .Select(n => n.ConsultaId)
            .ToHashSet();

        var consultas = dados.Consultas.ListarTodos()
            .Where(c => c.Status == StatusConsulta.Agendada
                        && !jaLembradas.Contains(c.Id)
                        && dados.Horarios.ObterPorId(c.HorarioId)!.Data == dia)
            .ToList();

        foreach (var consulta in consultas)
            Registrar(consulta, TipoNotificacao.Lembrete);

        return consultas.Count;
    }

    public IEnumerable<Notificacao> ListarPendentes() =>
        dados.Notificacoes.ListarTodos().Where(n => n.Status == StatusNotificacao.Pendente);

    public void MarcarComoEnviada(int notificacaoId)
    {
        var notificacao = dados.Notificacoes.ObterPorId(notificacaoId)
            ?? throw new InvalidOperationException("Notificação não encontrada.");
        notificacao.MarcarComoEnviada();
        dados.Notificacoes.Salvar();
    }

    private string MontarMensagem(Consulta consulta, TipoNotificacao tipo)
    {
        var horario = dados.Horarios.ObterPorId(consulta.HorarioId)!;
        var medico = dados.Medicos.ObterPorId(horario.MedicoId)!;
        var quando = $"{horario.Data:dd/MM/yyyy} às {horario.HoraInicio:HH:mm} com Dr(a). {medico.Nome}";

        var onde = horario.LocalId is int localId && dados.Locais.ObterPorId(localId) is { } local
            ? $" Local: {local.Nome}, {local.Endereco}, {local.Cidade}."
            : " Atendimento por teleconsulta.";

        return tipo switch
        {
            TipoNotificacao.Confirmacao => $"Consulta confirmada para {quando}.{onde}",
            TipoNotificacao.Reagendamento => $"Consulta reagendada para {quando}.{onde}",
            TipoNotificacao.Cancelamento => $"A consulta de {quando} foi cancelada.",
            TipoNotificacao.Lembrete => $"Lembrete: consulta em {quando}.{onde}",
            _ => throw new ArgumentOutOfRangeException(nameof(tipo))
        };
    }
}
```

MediCare.CsvApp/Entrada.cs

```
using System.Globalization;

namespace MediCare.CsvApp;

/// <summary>
/// Leitura de dados digitados no terminal.
/// </summary>
public static class Entrada
{
    private static readonly CultureInfo Cultura = CultureInfo.InvariantCulture;

    public static string? Ler(string rotulo)
    {
        Console.Write($"{rotulo}: ");
        return Console.ReadLine()?.Trim();
    }

    public static string LerTexto(string rotulo)
    {
        var valor = Ler(rotulo);
        if (string.IsNullOrWhiteSpace(valor))
            throw new ArgumentException($"{rotulo} é obrigatório.");
        return valor;
    }

    public static int LerInt(string rotulo) =>
        LerIntOpcional(rotulo) ?? throw new FormatException($"{rotulo} deve ser um número.");

    public static int? LerIntOpcional(string rotulo)
    {
        var texto = Ler(rotulo);
        if (string.IsNullOrWhiteSpace(texto))
            return null;
        return int.TryParse(texto, out var valor)
            ? valor
            : throw new FormatException($"{rotulo} deve ser um número.");
    }

    public static DateOnly LerData(string rotulo)
    {
        var texto = Ler($"{rotulo} (dd/mm/aaaa)");
        return DateOnly.TryParseExact(texto, "dd/MM/yyyy", Cultura, DateTimeStyles.None, out var data)
            ? data
            : throw new FormatException($"{rotulo} inválida.");
    }

    public static TimeOnly LerHora(string rotulo)
    {
        var texto = Ler($"{rotulo} (hh:mm)");
        return TimeOnly.TryParseExact(texto, "HH:mm", Cultura, DateTimeStyles.None, out var hora)
            ? hora
            : throw new FormatException($"{rotulo} inválida.");
    }

    public static TEnum Escolher<TEnum>(string rotulo) where TEnum : struct, Enum
    {
        var valores = Enum.GetValues<TEnum>();
        for (var i = 0; i < valores.Length; i++)
            Console.WriteLine($"  {i + 1} - {valores[i]}");

        var opcao = LerInt(rotulo);
        if (opcao < 1 || opcao > valores.Length)
            throw new ArgumentException($"{rotulo}: opção inválida.");
        return valores[opcao - 1];
    }
}
```

MediCare.CsvApp/Exibicao.cs

```
using MediCare.Core.Models;
using MediCare.Core.Repositories;

namespace MediCare.CsvApp;

/// <summary>
/// Monta os textos mostrados no terminal a partir dos dados carregados.
/// </summary>
public class Exibicao(MediCareDados dados)
{
    public void Medicos(IEnumerable<Medico> medicos)
    {
        foreach (var m in medicos)
            Console.WriteLine($"  [{m.Id}] {m.Nome} — {Especialidade(m.EspecialidadeId)} — {m.Crm}");
    }
    public void Especialidades()
```

```
    {
        foreach (var e in dados.Especialidades.ListarTodos())
            Console.WriteLine($"  [{e.Id}] {e.Nome}");
    }

    public void Horarios(IEnumerable<HorarioDisponivel> horarios)
    {
        var lista = horarios.ToList();
        if (lista.Count == 0)
            Console.WriteLine("  Nenhum horário encontrado.");
        foreach (var h in lista)
            Console.WriteLine($"  [{h.Id}] {Horario(h)}{(h.Disponivel ? "" : " (reservado)")}");
    }

    public void Consultas(IEnumerable<Consulta> consultas)
    {
        var lista = consultas.ToList();
        if (lista.Count == 0)
            Console.WriteLine("  Nenhuma consulta encontrada.");
        foreach (var c in lista)
            Console.WriteLine($"  [{c.Id}] {Consulta(c)}");
    }

    public string Consulta(Consulta c)
    {
        var horario = dados.Horarios.ObterPorId(c.HorarioId)!;
        var medico = dados.Medicos.ObterPorId(horario.MedicoId)!;
        var paciente = dados.Pacientes.ObterPorId(c.PacienteId)!;
        return $"{c.Status} — {paciente.Nome} com Dr(a). {medico.Nome} — {Horario(horario)}";
    }

    public string Horario(HorarioDisponivel h)
    {
        var texto = $"{h.Data:dd/MM/yyyy} {h.HoraInicio:HH:mm}-{h.HoraFim:HH:mm} — {h.Modalidade}";
        if (h.LocalId is int localId && dados.Locais.ObterPorId(localId) is { } local)
            texto += $" — {local.Nome}, {local.Endereco}, {local.Cidade}";
        return texto;
    }

    private string Especialidade(int id) => dados.Especialidades.ObterPorId(id)?.Nome ?? "?";
}
```

MediCare.CsvApp/Menu.cs

```
namespace MediCare.CsvApp;

public static class Menu
{
    /// <summary>
    /// Mostra as opções até o usuário escolher 0. Erros de regra de negócio
    /// ou de digitação são exibidos e o menu continua.
    /// </summary>
    public static void Executar(string titulo, params (string Texto, Action Acao)[] opcoes)
    {
        while (true)
        {
            Console.WriteLine($"\n== {titulo} ==");
            for (var i = 0; i < opcoes.Length; i++)
                Console.WriteLine($"{i + 1} - {opcoes[i].Texto}");
            Console.WriteLine("0 - Voltar");

            var texto = Entrada.Ler("Opção");
            if (texto is null or "0")
                return;

            if (!int.TryParse(texto, out var opcao) || opcao < 1 || opcao > opcoes.Length)
            {
                Console.WriteLine("Opção inválida.");
                continue;
            }

            try
            {
                opcoes[opcao - 1].Acao();
            }
            catch (Exception ex) when (ex is InvalidOperationException or ArgumentException or FormatException)
            {
                Console.WriteLine($"Não foi possível concluir: {ex.Message}");
            }
        }
    }
}
```

MediCare.CsvApp/Program.cs

```
using MediCare.Core.Models;
using MediCare.CsvApp;
using MediCare.CsvApp.Telas;

// Pasta com os arquivos CSV: primeiro argumento ou ./dados
var pasta = args.Length > 0 ? args[0] : "dados";

Sistema sistema;
try
{
    sistema = new Sistema(pasta);
}
catch (Exception ex) when (ex is InvalidDataException or IOException)
{
    Console.WriteLine($"Erro ao ler os arquivos CSV: {ex.Message}");
    return 1;
}

Console.WriteLine($"MediCare — dados carregados de '{Path.GetFullPath(pasta)}'");

Menu.Executar("MediCare",
    ("Entrar", Entrar),
    ("Criar conta de paciente", () => TelaCadastro.Paciente(sistema)),
    ("Criar conta de médico", () => TelaCadastro.Medico(sistema)));

return 0;

void Entrar()
{
    var usuario = sistema.Contas.Autenticar(Entrada.LerTexto("E-mail"), Entrada.LerTexto("Senha"));
    Console.WriteLine($"Bem-vindo(a), {usuario.Nome}.");

    switch (usuario)
    {
        case Paciente p: MenuPaciente.Abrir(sistema, p); break;
        case Recepcionista: MenuRecepcionista.Abrir(sistema); break;
        case Medico m: MenuMedico.Abrir(sistema, m); break;
        case Administrador: MenuAdministrador.Abrir(sistema); break;
    }
}
```

MediCare.CsvApp/Sistema.cs

```
using MediCare.Core.Repositories;
using MediCare.Core.Services;

namespace MediCare.CsvApp;

/// <summary>
/// Carrega os dados e cria os serviços usados pelos menus.
/// </summary>
public class Sistema
{
    public MediCareDados Dados { get; }
    public ContaService Contas { get; }
    public NotificacaoService Notificacoes { get; }
    public AgendamentoService Agendamento { get; }
    public DisponibilidadeService Disponibilidade { get; }
    public AtendimentoService Atendimento { get; }
    public Exibicao Exibir { get; }

    public Sistema(string pasta)
    {
        Dados = new MediCareDados(pasta);
        Contas = new ContaService(Dados);
        Notificacoes = new NotificacaoService(Dados);
        Agendamento = new AgendamentoService(Dados, Notificacoes);
        Disponibilidade = new DisponibilidadeService(Dados);
        Atendimento = new AtendimentoService(Dados);
        Exibir = new Exibicao(Dados);
    }
}
```

MediCare.CsvApp/Telas/FluxoAgendamento.cs

```
using MediCare.Core.Models;

namespace MediCare.CsvApp.Telas;

/// <summary>
/// Passos de agendamento comuns ao paciente e à recepcionista (UC03 a UC07).
/// </summary>
public class FluxoAgendamento(Sistema s)
{
```

```
    public void PesquisarMedicos()
    {
        s.Exibir.Especialidades();
        var especialidadeId = Entrada.LerIntOpcional("Especialidade (Enter para todas)");
        s.Exibir.Medicos(s.Agendamento.PesquisarMedicos(especialidadeId));
    }

    public void VerHorarios()
    {
        s.Exibir.Medicos(s.Agendamento.PesquisarMedicos());
        s.Exibir.Horarios(s.Agendamento.HorariosDisponiveis(Entrada.LerInt("Médico")));
    }

    public void Agendar(Paciente paciente)
    {
        s.Exibir.Medicos(s.Agendamento.PesquisarMedicos());
        s.Exibir.Horarios(s.Agendamento.HorariosDisponiveis(Entrada.LerInt("Médico")));

        var consulta = s.Agendamento.Agendar(paciente.Id, Entrada.LerInt("Horário"));
        Console.WriteLine($"Consulta {consulta.Id} agendada. Confirmação enviada por {paciente.CanalPreferido}.");
    }

    public void ListarConsultas(Paciente paciente) =>
        s.Exibir.Consultas(s.Agendamento.ConsultasDoPaciente(paciente.Id));

    public void Cancelar(Paciente paciente)
    {
        var consulta = EscolherConsultaAgendada(paciente);
        s.Agendamento.Cancelar(consulta.Id);
        Console.WriteLine("Consulta cancelada e horário liberado.");
    }

    public void Reagendar(Paciente paciente)
    {
        var consulta = EscolherConsultaAgendada(paciente);
        var medicoId = s.Dados.Horarios.ObterPorId(consulta.HorarioId)!.MedicoId;

        s.Exibir.Horarios(s.Agendamento.HorariosDisponiveis(medicoId));
        s.Agendamento.Reagendar(consulta.Id, Entrada.LerInt("Novo horário"));
        Console.WriteLine("Consulta reagendada.");
    }

    private Consulta EscolherConsultaAgendada(Paciente paciente)
    {
        var agendadas = s.Agendamento.ConsultasDoPaciente(paciente.Id)
            .Where(c => c.Status == StatusConsulta.Agendada)
            .ToList();
        s.Exibir.Consultas(agendadas);

        var id = Entrada.LerInt("Consulta");
        return agendadas.FirstOrDefault(c => c.Id == id)
            ?? throw new InvalidOperationException("Consulta não encontrada entre as agendadas do paciente.");
    }
}
```

MediCare.CsvApp/Telas/MenuAdministrador.cs

```
using MediCare.Core.Models;

namespace MediCare.CsvApp.Telas;

public static class MenuAdministrador
{
    public static void Abrir(Sistema s)
    {
        Menu.Executar("Administrador",
            ("Listar usuários", () => ListarUsuarios(s)),
            ("Ativar usuário", () => AlterarSituacao(s, ativo: true)),
            ("Desativar usuário", () => AlterarSituacao(s, ativo: false)),
            ("Listar agendamentos", () => ListarAgendamentos(s)),
            ("Cancelar agendamento", () => CancelarAgendamento(s)),
            ("Gerar lembretes de amanhã", () => GerarLembretes(s)),
            ("Enviar notificações pendentes", () => EnviarNotificacoes(s)));
    }

    private static void ListarUsuarios(Sistema s)
    {
        foreach (var u in s.Contas.ListarUsuarios())
        {
            var situacao = u.Ativo ? "ativo" : "desativado";
            Console.WriteLine($"  {u.GetType().Name,-14} {u.Nome,-25} {u.Email,-30} {situacao}");
        }
    }

    private static void AlterarSituacao(Sistema s, bool ativo)
```

```
    {
        s.Contas.AlterarSituacao(Entrada.LerTexto("E-mail do usuário"), ativo);
        Console.WriteLine(ativo ? "Usuário ativado." : "Usuário desativado.");
    }

    private static void ListarAgendamentos(Sistema s)
    {
        var filtro = Entrada.LerIntOpcional("Situação (1 Agendada, 2 Cancelada, 3 Realizada; Enter para todas)");
        StatusConsulta? status = filtro is int n && Enum.IsDefined(typeof(StatusConsulta), n - 1)
            ? (StatusConsulta)(n - 1)
            : null;
        s.Exibir.Consultas(s.Agendamento.ListarAgendamentos(status));
    }

    private static void CancelarAgendamento(Sistema s)
    {
        s.Exibir.Consultas(s.Agendamento.ListarAgendamentos(StatusConsulta.Agendada));
        s.Agendamento.Cancelar(Entrada.LerInt("Consulta"));
        Console.WriteLine("Consulta cancelada e horário liberado.");
    }

    private static void GerarLembretes(Sistema s)
    {
        var amanha = DateOnly.FromDateTime(DateTime.Today.AddDays(1));
        var quantidade = s.Notificacoes.GerarLembretes(amanha);
        Console.WriteLine($"{quantidade} lembrete(s) gerado(s) para {amanha:dd/MM/yyyy}.");
    }

    // O envio real depende do serviço externo (e-mail, SMS ou WhatsApp).
    // Aqui as mensagens são exibidas e marcadas como enviadas.
    private static void EnviarNotificacoes(Sistema s)
    {
        var pendentes = s.Notificacoes.ListarPendentes().ToList();
        if (pendentes.Count == 0)
            Console.WriteLine("  Nenhuma notificação pendente.");

        foreach (var n in pendentes)
        {
            Console.WriteLine($"  [{n.Canal} → {n.Destino}] {n.Mensagem}");
            s.Notificacoes.MarcarComoEnviada(n.Id);
        }
    }
}
```

MediCare.CsvApp/Telas/MenuMedico.cs

```
using MediCare.Core.Models;

namespace MediCare.CsvApp.Telas;

public static class MenuMedico
{
    public static void Abrir(Sistema s, Medico medico)
    {
        Menu.Executar("Médico",
            ("Minha agenda", () => s.Exibir.Consultas(s.Atendimento.Agenda(medico.Id))),
            ("Registrar consulta realizada", () => Concluir(s, medico)),
            ("Histórico de um paciente", () => Historico(s, medico)),
            ("Meus horários", () => s.Exibir.Horarios(s.Disponibilidade.HorariosDoMedico(medico.Id))),
            ("Cadastrar horário", () => CadastrarHorario(s, medico)),
            ("Remover horário", () => RemoverHorario(s, medico)),
            ("Acessar teleconsulta", () => TelaTeleconsulta.Acessar(s, medico)),
            ("Atualizar telefone", () => AtualizarPerfil(s, medico)));
    }

    private static void Concluir(Sistema s, Medico medico)
    {
        s.Exibir.Consultas(s.Atendimento.Agenda(medico.Id));
        s.Atendimento.ConcluirConsulta(medico.Id, Entrada.LerInt("Consulta"));
        Console.WriteLine("Consulta registrada como realizada.");
    }

    private static void Historico(Sistema s, Medico medico)
    {
        var paciente = s.Contas.ObterPorEmail(Entrada.LerTexto("E-mail do paciente")) as Paciente
            ?? throw new InvalidOperationException("Paciente não encontrado.");
        s.Exibir.Consultas(s.Atendimento.HistoricoDoPaciente(medico.Id, paciente.Id));
    }

    private static void CadastrarHorario(Sistema s, Medico medico)
    {
        var data = Entrada.LerData("Data");
        var inicio = Entrada.LerHora("Início");
        var fim = Entrada.LerHora("Término");
        var modalidade = Entrada.Escolher<Modalidade>("Modalidade");
```

```
        int? localId = null;
        if (modalidade == Modalidade.Presencial)
        {
            foreach (var l in s.Disponibilidade.LocaisDoMedico(medico.Id))
                Console.WriteLine($"  [{l.Id}] {l.Nome}, {l.Endereco}, {l.Cidade}");
            localId = Entrada.LerInt("Local");
        }

        var horario = s.Disponibilidade.CadastrarHorario(medico.Id, data, inicio, fim, modalidade, localId);
        Console.WriteLine($"Horário {horario.Id} cadastrado.");
    }

    private static void AtualizarPerfil(Sistema s, Medico medico)
    {
        Console.WriteLine($"Telefone atual: {medico.Telefone}");
        s.Contas.AtualizarPerfil(medico, Entrada.LerTexto("Novo telefone"));
        Console.WriteLine("Dados atualizados.");
    }

    private static void RemoverHorario(Sistema s, Medico medico)
    {
        s.Exibir.Horarios(s.Disponibilidade.HorariosDoMedico(medico.Id));
        s.Disponibilidade.RemoverHorario(medico.Id, Entrada.LerInt("Horário"));
        Console.WriteLine("Horário removido.");
    }
}
```

MediCare.CsvApp/Telas/MenuPaciente.cs

```
using MediCare.Core.Models;

namespace MediCare.CsvApp.Telas;

public static class MenuPaciente
{
    public static void Abrir(Sistema s, Paciente paciente)
    {
        var fluxo = new FluxoAgendamento(s);

        Menu.Executar("Paciente",
            ("Pesquisar médicos", fluxo.PesquisarMedicos),
            ("Ver horários de um médico", fluxo.VerHorarios),
            ("Agendar consulta", () => fluxo.Agendar(paciente)),
            ("Minhas consultas", () => fluxo.ListarConsultas(paciente)),
            ("Cancelar consulta", () => fluxo.Cancelar(paciente)),
            ("Reagendar consulta", () => fluxo.Reagendar(paciente)),
            ("Acessar teleconsulta", () => TelaTeleconsulta.Acessar(s, paciente)),
            ("Atualizar perfil", () => AtualizarPerfil(s, paciente)));
    }

    private static void AtualizarPerfil(Sistema s, Paciente paciente)
    {
        Console.WriteLine($"Telefone atual: {paciente.Telefone} — canal atual: {paciente.CanalPreferido}");
        s.Contas.AtualizarPerfil(paciente,
            Entrada.LerTexto("Novo telefone"),
            Entrada.Escolher<CanalNotificacao>("Canal para receber avisos"));
        Console.WriteLine("Dados atualizados.");
    }
}
```

MediCare.CsvApp/Telas/MenuRecepcionista.cs

```
using MediCare.Core.Models;

namespace MediCare.CsvApp.Telas;

/// <summary>
/// A recepcionista executa os mesmos passos do paciente, informando para qual paciente está atendendo.
/// </summary>
public static class MenuRecepcionista
{
    public static void Abrir(Sistema s)
    {
        var fluxo = new FluxoAgendamento(s);

        Menu.Executar("Recepção",
            ("Pesquisar médicos", fluxo.PesquisarMedicos),
            ("Ver horários de um médico", fluxo.VerHorarios),
            ("Agendar consulta para paciente", () => fluxo.Agendar(EscolherPaciente(s))),
            ("Consultas de um paciente", () => fluxo.ListarConsultas(EscolherPaciente(s))),
            ("Cancelar consulta de paciente", () => fluxo.Cancelar(EscolherPaciente(s))),
            ("Reagendar consulta de paciente", () => fluxo.Reagendar(EscolherPaciente(s))));
    }
    private static Paciente EscolherPaciente(Sistema s)
```

```
    {
        var paciente = s.Contas.ObterPorEmail(Entrada.LerTexto("E-mail do paciente")) as Paciente
            ?? throw new InvalidOperationException("Paciente não encontrado.");
        Console.WriteLine($"Paciente: {paciente.Nome}");
        return paciente;
    }
}
```

MediCare.CsvApp/Telas/TelaCadastro.cs

```
using MediCare.Core.Models;

namespace MediCare.CsvApp.Telas;

/// <summary>
/// UC01 — Criar conta.
/// </summary>
public static class TelaCadastro
{
    public static void Paciente(Sistema s)
    {
        var paciente = s.Contas.CadastrarPaciente(
            Entrada.LerTexto("Nome"),
            Entrada.LerTexto("E-mail"),
            Entrada.LerTexto("Senha"),
            Entrada.LerTexto("Telefone"),
            Entrada.LerData("Data de nascimento"),
            Entrada.Escolher<CanalNotificacao>("Canal para receber avisos"));

        Console.WriteLine($"Conta criada. Seu código de paciente é {paciente.Id}.");
    }

    public static void Medico(Sistema s)
    {
        var nome = Entrada.LerTexto("Nome");
        var email = Entrada.LerTexto("E-mail");
        var senha = Entrada.LerTexto("Senha");
        var crm = Entrada.LerTexto("CRM");
        var telefone = Entrada.LerTexto("Telefone");
        s.Exibir.Especialidades();
        var especialidadeId = Entrada.LerInt("Especialidade");

        var medico = s.Contas.CadastrarMedico(nome, email, senha, crm, telefone, especialidadeId);
        Console.WriteLine($"Conta criada. Seu código de médico é {medico.Id}.");
    }
}
```

MediCare.CsvApp/Telas/TelaTeleconsulta.cs

```
using MediCare.Core.Models;

namespace MediCare.CsvApp.Telas;

/// <summary>
/// UC11 — Realizar teleconsulta. O sistema libera o acesso ao paciente e ao médico da consulta
/// no horário agendado; a chamada de vídeo é feita pela plataforma de teleconsulta.
/// </summary>
public static class TelaTeleconsulta
{
    public static void Acessar(Sistema s, Usuario usuario)
    {
        s.Exibir.Consultas(s.Atendimento.TeleconsultasAgendadas(usuario));
        var consulta = s.Atendimento.AcessarTeleconsulta(usuario, Entrada.LerInt("Consulta"), DateTime.Now);
        Console.WriteLine($"Acesso liberado: {s.Exibir.Consulta(consulta)}");
    }
}
```